СОДЕРЖАНИЕ

ВВЕДЕНИЕ....2
1. ТЕОРЕТИЧЕСКИЕ И ТЕХНОЛОГИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ИНФОРМАЦИОННОЙ СИСТЕМЫ ДЛЯ УЧИТЕЛЯ МУЗЫКИ....5
1.1. Предметная область и роль информационных технологий в работе учителя музыки....5
1.2. Анализ существующих решений и постановка задачи....8
1.3. Обзор технологий и инструментов разработки....11
2. ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ СИСТЕМЫ-ПОМОЩНИКА УЧИТЕЛЯ МУЗЫКИ 14
2.1. Архитектура и структура программного продукта....14
2.2. Проектирование базы данных....18
2.3. Реализация основных функциональных модулей....21
2.4. Разграничение прав доступа и защита информации....28
2.5. Тестирование и оценка качества разработанной системы....31
ЗАКЛЮЧЕНИЕ....35
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ И ЛИТЕРАТУРЫ....37
ПРИЛОЖЕНИЕ А....40
ПРИЛОЖЕНИЕ Б....44
ПРИЛОЖЕНИЕ В....45

ВВЕДЕНИЕ

Современный этап развития школьного образования характеризуется активным внедрением цифровых технологий во все направления деятельности педагога. Электронные журналы, образовательные платформы, облачные хранилища и сервисы подготовки материалов стали привычной частью работы учителя. При этом степень автоматизации отдельных предметных областей остаётся неодинаковой: если универсальные школьные системы в первую очередь ориентированы на учёт успеваемости и расписание, то специфические потребности учителей-предметников нередко остаются вне их функциональных возможностей.

Особое место в этом отношении занимает работа учителя музыки. Она объединяет урочную и внеурочную деятельность, индивидуальные и групповые формы занятий, подготовку концертов и конкурсов, работу с репертуаром, создание планов-конспектов, подбор аудио- и видеоматериалов, ведение журнала посещаемости и оценивания. Значительная часть этих сведений хранится в разрозненных документах: бумажном журнале, рабочих программах, папках с файлами, электронных таблицах и переписке. В результате учитель вынужден многократно переносить одни и те же данные из одного документа в другой, тратить время на поиск нужного материала и вручную составлять отчёты.

Актуальность темы курсовой работы обусловлена необходимостью создания единого информационного пространства, которое объединяет основные направления работы учителя музыки и снижает долю рутинных операций. Федеральный государственный образовательный стандарт основного общего образования и профессиональный стандарт педагога прямо указывают на необходимость применения информационно-коммуникационных технологий в планировании, проведении и оценке результатов учебной деятельности. Вместе с тем универсальные образовательные платформы, как правило, не содержат специализированных инструментов для работы с музыкальным репертуаром, конспектами уроков музыки и материалами творческих мероприятий. Следовательно, разработка компактной предметно-ориентированной системы, учиты-

вающей особенности музыкального образования, является практически значимой задачей.

Цель курсовой работы заключается в разработке веб-приложения «Система-помощник учителя музыки в школе», обеспечивающего автоматизацию ведения классов и учеников, расписания, репертуара, планов-конспектов, материалов, журнала, мероприятий и отчётов, а также разграничение прав доступа для учителя, администратора и родителя.

Для достижения поставленной цели необходимо решить следующие задачи:

– изучить предметную область и нормативные требования, определяющие организацию работы учителя музыки в школе;

– проанализировать существующие информационные решения и выявить их возможности и ограничения применительно к музыкальному образованию;

– сформулировать функциональные и нефункциональные требования к проектируемой системе, определить роли пользователей и сценарии работы;

– выбрать технологии и инструменты разработки, обосновать архитектурные решения;

– спроектировать структуру веб-приложения и модель базы данных;

– реализовать основные функциональные модули и механизм разграничения прав доступа;

– провести тестирование разработанной системы и оценить результаты её опытной эксплуатации.

Объектом исследования выступает процесс организации учебной, методической и внеурочной работы учителя музыки в общеобразовательной школе.

Предметом исследования являются программные средства и информационные технологии автоматизации деятельности учителя музыки, а также методы проектирования и реализации веб-приложений, обеспечивающих хранение, обработку и представление профессионально значимых данных.

При выполнении работы использовались следующие методы: анализ научной, учебной и нормативной литературы; сравнительный анализ существующих программных решений; системное и объектно-ориентированное моделирование; проектирование баз данных; программная реализация; автоматизированное и ручное тестирование.

Практическая значимость работы состоит в том, что разработанное приложение представляет собой законченный программный продукт, который может быть развёрнут в школе без приобретения дополнительных лицензий. Система содержит демонстрационные данные, руководство пользователя и автоматические тесты, что упрощает её внедрение и дальнейшее сопровождение. Использование единой базы данных позволяет сократить время на подготовку отчётов, обеспечить сохранность методических материалов и предоставить родителям прозрачный доступ к успеваемости и посещаемости их детей.

Структура курсовой работы обусловлена целью и задачами исследования и включает введение, две главы, объединяющие восемь параграфов, заключение, список использованных источников и литературы, а также приложения. В первой главе рассматриваются предметная область, существующие решения, требования к системе и обоснование выбора технологий. Во второй главе описываются архитектура приложения, модель базы данных, реализация функциональных модулей, механизм разграничения прав доступа, а также результаты тестирования.

1. ТЕОРЕТИЧЕСКИЕ И ТЕХНОЛОГИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ИНФОРМАЦИОННОЙ СИСТЕМЫ ДЛЯ УЧИТЕЛЯ МУЗЫКИ

1.1. Предметная область и роль информационных технологий в работе учителя музыки

Учебный предмет «Музыка» занимает особое положение в системе общего образования, поскольку его содержание связано не только с усвоением знаний, но и с развитием эмоционально-ценностного отношения к искусству, вокально-хоровых умений, слушательской культуры и творческих способностей школьников. В соответствии с федеральным государственным образовательным стандартом основного общего образования предметная область «Искусство» ориентирована на формирование художественной культуры обучающихся, воспитание эстетического вкуса и приобретение опыта творческой деятельности. Эти задачи решаются как на уроках, так и во внеурочной работе: в хоровых студиях, вокальных ансамблях, школьных концертах, конкурсах и музыкальных гостиных.

Профессиональная деятельность учителя музыки включает несколько взаимосвязанных направлений. Во-первых, это преподавание в параллелях и классах, каждый из которых имеет свою программу, тематическое планирование и особенности состава обучающихся. Во-вторых, это работа с музыкальным репертуаром: подбор произведений для слушания, разучивания и исполнения с учётом возраста, вокальных возможностей и уровня подготовки детей. В-третьих, это методическая работа — разработка планов-конспектов, технологических карт уроков, критериев оценивания и домашних заданий. В-четвёртых, это организация творческих мероприятий и взаимодействие с классными руководителями, администрацией и родителями.

Каждое из перечисленных направлений порождает собственный поток информации. Сведения об учениках и их родителях, расписание уроков, темы занятий, посещаемость, оценки, репертуар, ссылки на аудиозаписи и видеоматериалы, тексты конспектов и сценарии мероприятий хранятся в разных форматах и, как правило, не связаны между собой. Учитель вынужден вести

бумажный журнал, отдельно готовить отчёты в текстовом редакторе или электронных таблицах, отдельно хранить файлы на рабочем компьютере или в облачном диске. При переходе к другому классу или параллели часть данных приходится вводить повторно.

Нормативную основу рассматриваемой предметной области образуют несколько документов. Федеральный закон от 29.12.2012 № 273-ФЗ «Об образовании в Российской Федерации» закрепляет право образовательных организаций использовать электронное обучение и дистанционные образовательные технологии, а также требования к информационной открытости образовательной организации. Федеральный государственный образовательный стандарт основного общего образования определяет планируемые результаты освоения предметной области «Искусство», включая развитие музыкальной культуры и творческих способностей обучающихся. Профессиональный стандарт педагога относит к трудовым действиям учителя планирование и проведение учебных занятий, формирование универсальных учебных действий, использование современных информационно-коммуникационных технологий и объективную оценку результатов обучения.

Таким образом, применение цифровых инструментов в работе учителя музыки является не дополнительным преимуществом, а требованием времени и нормативных документов. Однако простое увеличение количества используемых программ не решает проблему: разрозненные сервисы лишь усиливают фрагментацию данных. Необходима единая среда, в которой сведения об учениках, уроках, репертуаре, материалах и результатах обучения связаны между собой и доступны через понятный интерфейс.

Анализ типичных затруднений учителя музыки позволяет выделить несколько ключевых проблем. Первая связана с многократным дублированием информации: список класса приходится переносить в журнал, затем в отчёт, затем в сценарий мероприятия. Вторая проблема — потеря и затруднённый поиск методических материалов: файлы с нотами, фонограммами и презентациями хранятся в папках с произвольными именами. Третья проблема — высокая тру-

доёмкость отчётности: подсчёт посещаемости и среднего балла выполняется вручную, что увеличивает риск ошибок. Четвёртая проблема — ограниченный доступ родителей к информации об успеваемости ребёнка: сведения передаются через классного руководителя или родительские собрания и не всегда оперативны.

Информационные технологии позволяют устранить или существенно ослабить перечисленные проблемы. Единая база данных обеспечивает однократный ввод и многократное использование сведений. Автоматизированные запросы и отчёты избавляют от ручного подсчёта. Полнотекстовый поиск по названиям, авторам и описаниям ускоряет подготовку к уроку. Веб-интерфейс делает систему доступной с любого устройства, имеющего браузер, без установки дополнительного программного обеспечения. Разграничение прав доступа позволяет предоставить каждому участнику образовательного процесса именно те сведения, которые необходимы для его работы.

Вместе с тем автоматизация должна учитывать специфику музыкального образования. Универсальные электронные журналы хорошо справляются с учётом оценок и посещаемости, но не предназначены для хранения репертуара, описания характера исполнения, ссылок на аудиозаписи и видеозаписи выступлений. Планы-конспекты уроков музыки имеют свою структуру: цель, задачи, музыкальный материал, оборудование, этапы слушания и вокально-хоровой работы, критерии оценивания. Материалы к занятиям включают не только документы и презентации, но и аудиофайлы, MIDI-файлы, изображения нотных примеров. Следовательно, предметно-ориентированная система должна поддерживать указанные типы данных и соответствующие сценарии работы.

Отдельного внимания заслуживает вопрос о связи школы и семьи. Родители заинтересованы в получении своевременной информации об успеваемости и посещаемости своего ребёнка, о предстоящих мероприятиях и необходимой подготовке. Предоставление таких сведений через персональный кабинет снижает нагрузку на учителя и классного руководителя, делает взаимодействие более прозрачным и упорядоченным. При этом доступ должен быть строго

ограничен данными собственных детей, что предъявляет повышенные требования к механизму авторизации.

Таким образом, предметная область характеризуется разнообразием взаимосвязанных данных, высокой долей рутинных операций и необходимостью соблюдения нормативных требований к информационной открытости и защите персональных данных. Эти особенности определяют облик будущей системы: она должна быть компактной, предметно-ориентированной, доступной через браузер и поддерживать роли учителя, администратора и родителя.

1.2. Анализ существующих решений и постановка задачи

Для обоснования необходимости собственной разработки был проведён сравнительный анализ программных продуктов и сервисов, которые могут использоваться учителем музыки. В выборку вошли универсальные школьные информационные системы, платформы дистанционного обучения, облачные офисные сервисы и специализированные музыкальные приложения. Критериями сравнения стали наличие журнала и оценок, поддержка репертуара и планов-конспектов, работа с файлами и медиаматериалами, формирование отчётов, разграничение ролей, а также сложность внедрения и стоимость.

Электронные журналы и дневники, такие как «Дневник.ру» и «ЭлЖур», являются наиболее распространёнными решениями в российских школах. Они обеспечивают учёт успеваемости, посещаемости, домашних заданий и позволяют родителям видеть оценки ребёнка. Однако их функциональность ориентирована на общешкольный документооборот и не включает специализированные разделы для музыкального репертуара, конспектов уроков музыки и творческих мероприятий. Кроме того, внедрение такой системы требует административного решения на уровне школы и не может быть выполнено отдельным учителем.

Платформа Moodle представляет собой полноценную систему управления обучением. Она поддерживает курсы, задания, тесты, форумы, загрузку файлов и развёрнутое оценивание. Вместе с тем Moodle избыточна для задач одного учителя-предметника: её установка и настройка требуют администриро-

вания, интерфейс ориентирован на дистанционное обучение, а ведение классного журнала и отчётов по посещаемости реализуется не так удобно, как в специализированных школьных системах. Значительная часть возможностей платформы остаётся невостребованной.

Сервис Google Classroom удобен для выдачи заданий и обмена материалами, но не является журналом в понимании российской школы. В нём отсутствуют привычные отчёты по посещаемости и среднему баллу, а работа зависит от наличия учётных записей и доступа к внешним сервисам. Облачные таблицы и документы позволяют создать произвольную структуру учёта, однако не обеспечивают связность данных, разграничение прав и удобный поиск; при росте объёма сведений такие файлы быстро становятся трудно поддерживаемыми.

Специализированные музыкальные приложения и тренажёры, например сервисы для развития слуха, нотной грамоты и игры на инструменте, решают узкие образовательные задачи. Они не предназначены для ведения классов, журнала, отчётов и методической документации. Таким образом, на рынке присутствуют либо универсальные, но функционально избыточные системы, либо узкие инструменты, не покрывающие потребности учителя музыки в комплексе.

Результаты сравнения обобщены в таблице 1.

Таблица 1 — Сравнение существующих решений

Решение	**Журнал и оценки**	**Репертуар и конспекты**	**Медиафайлы**	**Отчёты**	**Роли**	**Сложность внедрения**
Дневник.ру, ЭлЖур	Да	Нет	Частично	Да	Да	Высокая, на уровне школы
Moodle	Частично	Нет	Да	Частично	Да	Высокая, требуется администрирование
Google Classroom	Нет	Нет	Да	Нет	Частично	Низкая, но зависит от внешнего сервиса
Облачные таблицы	Вручную	Нет	Нет	Вручную	Нет	Низкая, но данные не связаны
Музыкальные тренажёры	Нет	Нет	Частично	Нет	Нет	Низкая, узкое назначение
Разрабатываемая система	Да	Да	Да	Да	Да	Низкая, развёртывание в школе

Анализ таблицы 1 показывает, что ни одно из рассмотренных решений не покрывает одновременно журнал, репертуар, планы-конспекты, медиаматериалы, отчёты и разграничение ролей при низкой сложности внедрения. Это обосновывает целесообразность разработки собственного веб-приложения, ориентированного на конкретного пользователя — учителя музыки — и не требующего общешкольной информационной инфраструктуры.

На основе изучения предметной области были сформулированы функциональные требования к системе. Они представлены в таблице 2.

Таблица 2 — Функциональные требования к системе

Код	Требование	Приоритет
F1	Аутентификация пользователей и разграничение ролей (администратор, учитель, родитель)	Высокий
F2	Ведение справочника классов и учеников с привязкой к учителю и родителю	Высокий
F3	Составление расписания уроков по дням недели с проверкой времени	Высокий
F4	Ведение репертуара с указанием композитора, жанра, сложности и ссылок	Средний
F5	Создание планов-конспектов уроков по заданной структуре	Средний
F6	Загрузка материалов и файлов с проверкой типа и размера	Высокий
F7	Ведение журнала посещаемости и оценок без дублирования записей	Высокий
F8	Формирование отчётов по классу и ученику, экспорт в CSV	Средний
F9	Ведение афиши мероприятий и концертов	Средний
F10	Глобальный поиск по доступным пользователю данным	Средний
F11	Личный кабинет родителя с фильтрацией по своим детям	Высокий
F12	Переключение светлой и тёмной темы интерфейса	Низкий

К нефункциональным требованиям отнесены: удобство освоения интерфейса без специальной подготовки; адаптивность вёрстки для планшетов и телефонов; время отклика страницы в локальной сети не более одной секунды; сохранность данных при перезапуске приложения; возможность работы под управлением Windows без установки СУБД; переносимость на MySQL при росте нагрузки; наличие автоматических тестов и документации.

Границы системы определены сознательно. Приложение не претендует на роль общешкольного электронного дневника: оно не ведёт учёт по всем предметам, не выполняет функции бухгалтерии, не поддерживает видеоконференции и не заменяет официальные документы образовательной организации. Система предназначена для внутренней организации работы учителя музыки и

информирования родителей по предмету «Музыка». Такое ограничение позволяет сохранить простоту и обеспечить высокое качество реализации ключевых сценариев.

Критериями успешности разработки являются: полное покрытие требований F1–F12; отсутствие ошибок при прохождении автоматических тестов; корректная работа разграничения доступа при попытке открыть чужие данные по прямому адресу; удобство выполнения основных операций; наличие демонстрационных данных и руководства пользователя.

1.3. Обзор технологий и инструментов разработки

Выбор технологий разработки определялся требованиями к простоте развёртывания, скорости создания прототипа, надёжности и сопровождаемости. Поскольку система должна работать в школе на обычном компьютере под управлением Windows, предпочтение было отдано решениям, не требующим сложной инфраструктуры и платных лицензий. В качестве основного языка программирования выбран Python, обладающий выразительным синтаксисом, развитой стандартной библиотекой и широкой экосистемой веб-фреймворков.

Python позволяет сосредоточиться на логике предметной области, а не на технических деталях. Большое количество библиотек для работы с базами данных, формами, шаблонами и тестированием сокращает объём собственного кода. Кроссплатформенность языка обеспечивает возможность запуска как на Windows, так и на Linux-сервере, что важно для дальнейшего развития проекта.

Среди веб-фреймворков Python наиболее известны Django и Flask. Django представляет собой полнофункциональный фреймворк с встроенной административной панелью, ORM, системой миграций и строгой структурой проекта. Flask, напротив, является микрофреймворком: он предоставляет минимальный набор средств для маршрутизации и работы с шаблонами, а остальные компоненты подключаются по мере необходимости. Для учебного проекта, ограниченного предметной областью одного учителя, Flask оказался предпочтительнее, поскольку позволяет построить понятную и компактную архитектуру без избыточных абстракций.

Работа с базой данных организована через SQLAlchemy — библиотеку объектно-реляционного отображения, входящую в состав Flask-SQLAlchemy. Модели описываются в виде классов Python, что делает схему данных наглядной и позволяет избежать ручного написания SQL-запросов. Использование ORM снижает риск SQL-инъекций и упрощает переход между СУБД. В качестве основной СУБД выбрана SQLite: она не требует установки отдельного сервера, хранит данные в одном файле и полностью подходит для небольшой школы. При необходимости приложение может быть переключено на MySQL изменением строки подключения, поскольку драйвер PyMySQL включён в зависимости.

Аутентификация и авторизация реализованы с помощью Flask-Login. Расширение управляет сессиями, предоставляет объект текущего пользователя и декораторы для защиты маршрутов. Пароли хранятся не в открытом виде, а в виде хешей, сформированных функцией generate_password_hash библиотеки Werkzeug. Для защиты форм от подделки межсайтовых запросов используется Flask-WTF с глобально включённым CSRFProtect: каждый POST-запрос должен содержать действительный токен.

Серверная генерация HTML-страниц выполняется шаблонизатором Jinja2. Шаблоны наследуются от базового макета, что обеспечивает единообразие навигации, уведомлений и оформления. Автоматическое экранирование переменных в Jinja2 служит дополнительной защитой от XSS-атак. Для оформления интерфейса применён фреймворк Bootstrap 5: он предоставляет адаптивную сетку, готовые компоненты форм, таблиц, модальных окон и навигации. Поверх Bootstrap реализована собственная тема «тёмный концертный зал» с деликатными свечениями, полупрозрачными карточками и переключателем светлой и тёмной темы.

В качестве WSGI-сервера используется waitress. В отличие от встроенного сервера Flask, предназначенного для разработки, waitress является production-сервером, работает под Windows и не требует дополнительных настроек. Это позволяет запускать приложение в школе одной командой. Для

автоматического тестирования выбран pytest: он поддерживает фикстуры, параметризацию и удобный вывод результатов, а также хорошо сочетается с фабрикой приложения Flask.

Управление версиями осуществляется с помощью Git, удалённый репозиторий размещён на GitHub. Такой подход обеспечивает сохранность истории изменений, возможность отката и наглядное документирование этапов разработки. Среда разработки — Visual Studio Code, виртуальное окружение Python создаётся средствами модуля venv. Перечень используемых технологий приведён в таблице 3.

Таблица 3 — Технологический стек проекта

Уровень	Технология	Назначение
Язык программирования	Python 3.10+	серверная логика приложения
Веб-фреймворк	Flask 3.0	маршрутизация, сессии, шаблоны
ORM	SQLAlchemy 2.x, Flask-SQLAlchemy	работа с базой данных через модели
СУБД	SQLite (MySQL — опционально)	хранение данных
Аутентификация	Flask-Login, Werkzeug	вход, сессии, хеширование паролей
Формы и защита	Flask-WTF, CSRFProtect	валидация и защита POST-запросов
Шаблоны	Jinja2	генерация HTML-страниц
Интерфейс	Bootstrap 5, Bootstrap Icons, Manrope	адаптивная вёрстка и оформление
Файлы	локальная папка uploads	хранение материалов к урокам
WSGI-сервер	waitress	запуск в режиме эксплуатации
Тестирование	pytest	автоматическая проверка функциональности
Версии	Git, GitHub	хранение и история изменений

Совокупность выбранных технологий образует полный стек разработки веб-приложения: от языка и ORM до сервера и инструментов тестирования. Все компоненты являются свободно распространяемыми, хорошо документированы и широко используются в учебных и коммерческих проектах. Это снижает риски, связанные с освоением инструментов, и обеспечивает возможность дальнейшего развития системы силами студентов и преподавателей.

2. ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ СИСТЕМЫ-ПОМОЩНИКА УЧИТЕЛЯ МУЗЫКИ

2.1. Архитектура и структура программного продукта

Архитектура приложения построена по принципу разделения ответственности между слоями. Такой подход позволяет изолировать работу с базой данных, бизнес-правила и представление, упрощает тестирование и дальнейшее развитие системы. В основе лежит схема, близкая к классической модели MVC: модели описывают данные, маршруты обрабатывают запросы и формируют контекст, а шаблоны отвечают за отображение. Дополнительно выделен слой правил доступа, который проверяет права пользователя до выполнения операции.

Приложение создаётся фабрикой create_app(), расположенной в модуле app/__init__.py. Фабрика инициализирует расширения (SQLAlchemy, Flask-Login, CSRFProtect), регистрирует Blueprints и обработчики ошибок, создаёт папку экземпляра и при необходимости заполняет базу демонстрационными данными. Использование фабрики важно для тестирования: каждый тест может создать собственный экземпляр приложения с временной базой данных, не затрагивая рабочую.

Функциональность разделена на одиннадцать Blueprints. Каждый Blueprint соответствует отдельному модулю и объединяет маршруты списка, создания, редактирования и удаления объектов. Такое разделение повторяет структуру предметной области и позволяет разрабатывать модули независимо. Общая схема взаимодействия компонентов представлена на рисунке 1.

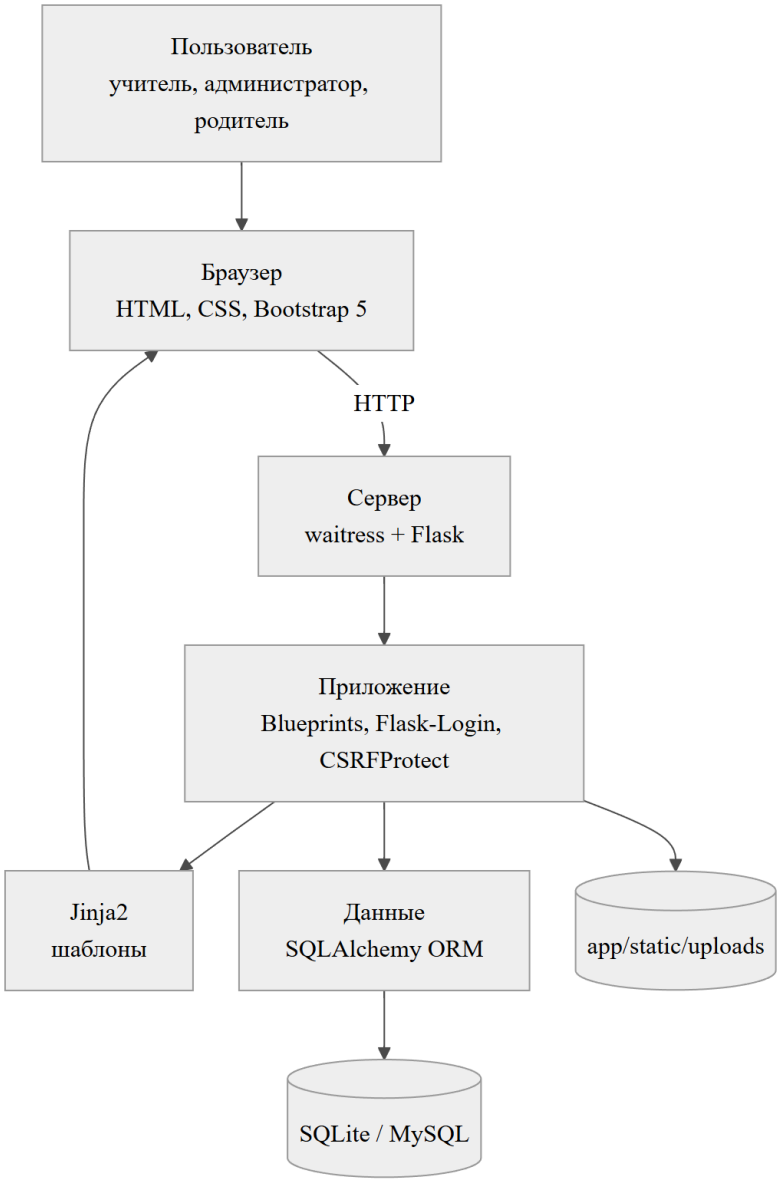


Рисунок 1 — Архитектура веб-приложения

Запрос пользователя поступает в браузер, передаётся по протоколу HTTP на WSGI-сервер waitress и далее в приложение Flask. Flask-Login определяет текущего пользователя и его роль, CSRFProtect проверяет токен для небезопасных методов, после чего управление передаётся соответствующему Blueprint. Маршрут обращается к моделям SQLAlchemy, формирует данные и передаёт их в шаблон Jinja2. Готовый HTML возвращается браузеру. Файлы материалов сохраняются в локальной папке uploads, а сведения о них — в базе данных.

Структура проекта организована следующим образом:

– app/__init__.py — фабрика приложения, инициализация расширений и Blueprints;

– app/models.py — десять моделей данных и константы предметной области;

– app/seed.py — демонстрационные данные для первого запуска;

– app/routes/ — маршруты модулей: auth, main, students, schedule, repertoire, lesson_plans, materials, journal, reports, events, search;

– app/templates/ — шаблоны Jinja2, сгруппированные по модулям;

– app/static/ — стили, клиентские скрипты и папка загруженных файлов;

– app/utils/ — декораторы проверки ролей и правила доступа к классам;

– tests/ — автоматические тесты pytest;

– config.py — конфигурация приложения;

– app.py — точка входа с запуском waitress;

– docs/ — проектная документация и материалы пояснительной записки.

Жизненный цикл запроса удобно рассматривать на примере журнала. При открытии страницы журнала пользователь передаёт идентификатор класса и дату. Flask-Login определяет роль, маршрут проверяет доступность класса, выполняет выборку посещаемости и оценок и передаёт данные в шаблон. При сохранении формы выполняется проверка CSRF-токена, затем маршрут создаёт или обновляет записи и возвращает перенаправление. Последовательность операций показана на рисунке 2.

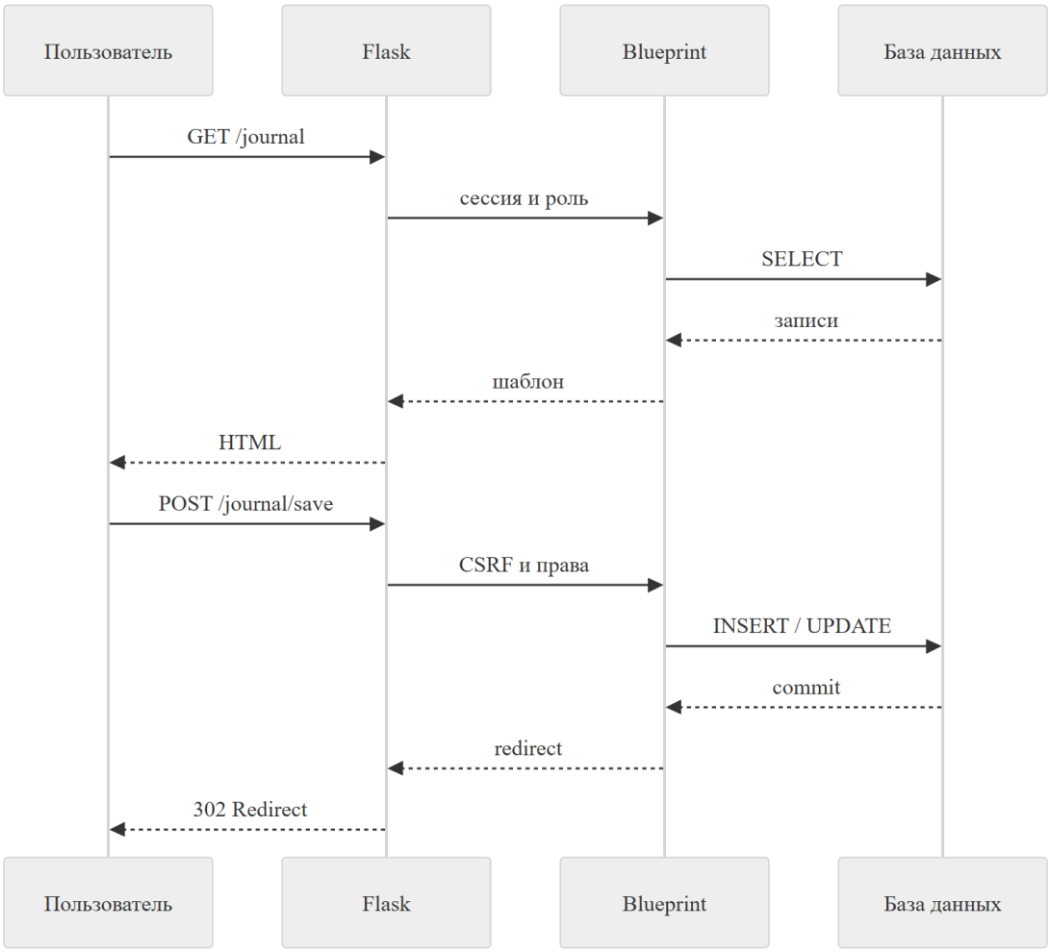


Рисунок 2 — Жизненный цикл запроса на примере журнала

Конфигурация вынесена в отдельный модуль config.py. Параметры подключения к базе данных, папка загрузок и секретный ключ могут задаваться через переменные окружения, что соответствует требованиям безопасности и упрощает развёртывание. Значения по умолчанию рассчитаны на локальный запуск: база данных размещается в папке instance, загрузки — в app/static/uploads, сервер слушает адрес 127.0.0.1 и порт 5000.

Обработка ошибок реализована централизованно. Для кодов 400, 403, 404, 413 и 500 предусмотрены собственные страницы с понятными сообщениями и ссылкой возврата. В частности, ошибка 400 возникает при отсутствии или истечении CSRF-токена, ошибка 403 — при недостаточных правах, ошибка 413 — при превышении допустимого размера файла. Централизованная обработка исключает показ технических трассировок пользователю и делает поведение системы предсказуемым.

Архитектура допускает горизонтальное развитие. Новый функциональный модуль оформляется как отдельный Blueprint и регистрируется в фабрике приложения; модель данных добавляется в models.py; шаблоны размещаются в соответствующей папке. Переход на MySQL выполняется изменением строки подключения без изменения кода моделей и маршрутов. Такая расширяемость важна для дальнейшего сопровождения проекта.

2.2. Проектирование базы данных

Модель данных спроектирована на основе выделенных сущностей предметной области. В системе используются десять таблиц: пользователи, классы, ученики, расписание, репертуар, материалы, мероприятия, посещаемость, оценки и планы-конспекты. Каждая таблица имеет первичный ключ, а связи между таблицами реализованы через внешние ключи. Связи ядра базы данных показаны на рисунке 3, связи учебных сущностей — на рисунке 4.

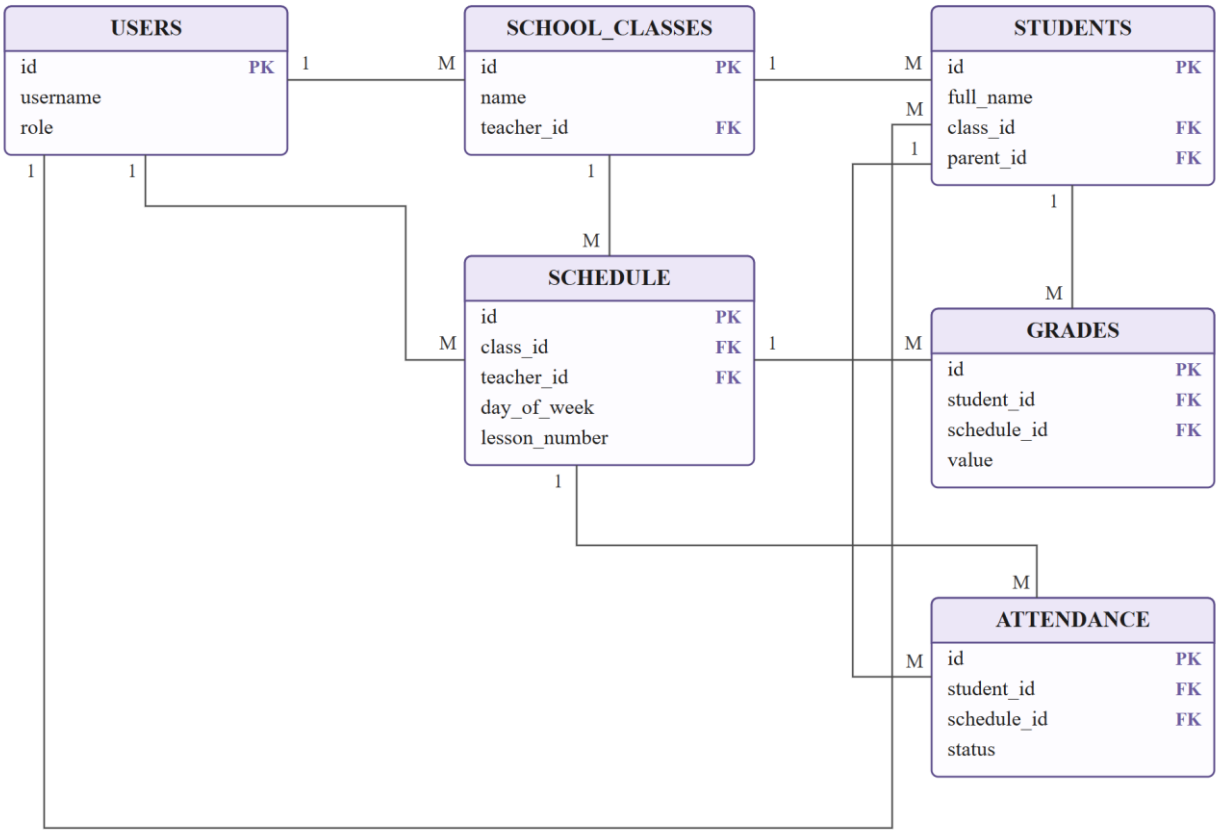


Рисунок 3 — ER-модель: пользователи, классы, ученики и журнал

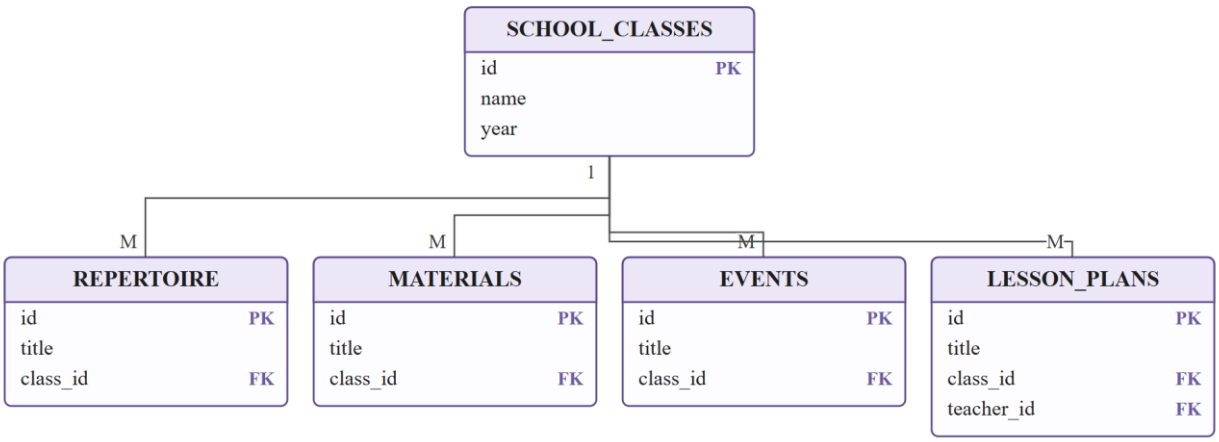


Рисунок 4 — ER-модель: учебные сущности

Таблица users хранит учётные записи всех пользователей независимо от роли. Поле role принимает значения admin, teacher или parent и определяет доступные пользователю разделы. Логин и адрес электронной почты объявлены уникальными, пароль хранится в виде хеша. Таблица school_classes описывает классы с указанием параллели, учебного года и учителя. Таблица students связывает ученика с классом и, при необходимости, с родителем. Такое решение позволяет одному родителю иметь несколько детей в разных классах.

Таблица schedule содержит уроки с привязкой к классу и учителю, днём недели, номером урока, временем и кабинетом. Таблица repertoire предназначена для произведений с указанием композитора, жанра, сложности, длительности и ссылки на аудио- или видеозапись. Поле class_id в репертуаре может быть пустым: в этом случае произведение считается общим для всех классов. Аналогичное правило действует для материалов и мероприятий, что позволяет не дублировать общие ресурсы.

Таблица materials хранит сведения о методических материалах: название, описание, тип занятия, класс, имя и путь к файлу, внешнюю ссылку. Физический файл размещается в папке uploads, а в базе сохраняется относительный путь. Таблица events описывает афишу мероприятий с датой, временем, местом и статусом. Таблица lesson_plans содержит структурированный конспект урока: тему, раздел программы, цель, задачи, тип урока, оборудование, ход урока, домашнее задание и критерии оценивания.

Таблицы attendance и grades фиксируют результаты конкретного урока. Обе таблицы содержат ссылки на ученика, урок и дату, а также идентификатор учителя, выставившего запись. Посещаемость хранит статус (присутствовал, отсутствовал, опоздал, пропуск по уважительной причине) и примечание. Оценка содержит вид работы, значение от 1 до 5 и комментарий. Связь с уроком позволяет отличать записи разных занятий одного дня и строить корректные отчёты.

Назначение основных таблиц и их ключевые поля приведены в таблице 4.

Таблица 4 — Основные сущности базы данных

Таблица	Назначение	Ключевые поля
users	учётные записи и роли	id, username, email, password_hash, role
school_classes	классы школы	id, name, grade, year, teacher_id
students	ученики	id, full_name, class_id, parent_id
schedule	расписание уроков	id, class_id, teacher_id, day_of_week, lesson_number
repertoire	музыкальный репертуар	id, title, composer, genre, class_id
materials	материалы к урокам	id, title, class_id, file_path, link
events	мероприятия и концерты	id, title, event_date, place, status
attendance	посещаемость	id, student_id, schedule_id, lesson_date, status
grades	оценки	id, student_id, schedule_id, lesson_date, value
lesson_plans	планы-конспекты	id, title, class_id, lesson_date, structure

При проектировании соблюдались принципы нормализации. Справочные сведения (классы, ученики, пользователи) отделены от фактов (посещаемость, оценки), что исключает дублирование и аномалии обновления. Повторяющиеся текстовые значения, такие как статусы посещаемости и категории оценок, вынесены в константы моделей и используются в формах и шаблонах. Это снижает риск расхождений и упрощает изменение справочников.

Целостность данных обеспечивается на нескольких уровнях. На уровне базы данных действуют первичные ключи, ограничения NOT NULL и UNIQUE для логина, электронной почты и названия класса. На уровне ORM настроены каскадные операции: удаление класса приводит к удалению связанных учеников, а удаление ученика — к удалению его записей посещаемости и оценок. На

уровне маршрутов выполняется дополнительная проверка существования объектов и прав пользователя, что позволяет выводить понятные сообщения вместо технических ошибок.

Индексы предусмотрены для полей, по которым выполняется поиск и фильтрация: логин и электронная почта пользователя, идентификаторы классов и учеников, дата урока. Уникальные ограничения автоматически создают индексы. Для небольших объёмов данных SQLite не требует дополнительной настройки, однако заложенная схема без изменений переносится на MySQL, где те же индексы продолжают работать.

Создание схемы выполняется автоматически при первом запуске приложения с помощью метода db.create_all(). Демонстрационные данные формируются модулем seed.py: создаются пользователи трёх ролей, семь классов, ученики, расписание, репертуар, материалы, мероприятия и записи журнала. Наличие демонстрационного набора позволяет сразу оценить интерфейс и проверить сценарии без ручного ввода. При необходимости база данных может быть удалена и создана заново без изменения кода.

Выбранная модель данных обеспечивает решение всех функциональных требований. Она поддерживает связь между классами, учениками, уроками и результатами обучения, хранит методические материалы и репертуар, позволяет строить отчёты по классу и по отдельному ученику. При этом схема остаётся компактной и понятной, что важно для учебного проекта и дальнейшего сопровождения.

2.3. Реализация основных функциональных модулей

Приложение объединяет одиннадцать функциональных модулей, каждый из которых реализован отдельным Blueprint. Перечень модулей, их назначение и основные маршруты приведены в таблице 5.

Таблица 5 — Функциональные модули и маршруты

Модуль	Назначение	Основные маршруты
Авторизация	вход, выход, управление пользователями	/login, /logout, /users
Главная	рабочий стол учителя и кабинет родителя	/
Классы и ученики	справочники классов и учеников	/classes, /students
Расписание	сетка уроков по дням недели	/schedule
Репертуар	произведения и ссылки на записи	/repertoire
Планы-конспекты	структурированные конспекты уроков	/lesson-plans
Материалы	файлы, ссылки, типы занятий	/materials
Журнал	посещаемость, оценки, карточка ученика	/journal
Отчёты	сводка успеваемости и экспорт CSV	/reports
Мероприятия	афиша концертов и конкурсов	/events
Поиск	единая строка поиска по данным	/search

Главная страница выполняет роль рабочего стола. Для учителя она показывает сводную статистику: количество классов и учеников, ближайшие уроки, последние оценки, произведения репертуара и предстоящие мероприятия. Для родителя эта же страница преобразуется в личный кабинет: отображаются только его дети, их классы, расписание, последние оценки и мероприятия. Такое переключение контекста реализовано в одном маршруте с проверкой роли и фильтрацией выборок. Внешний вид рабочего стола учителя представлен на рисунке 5.

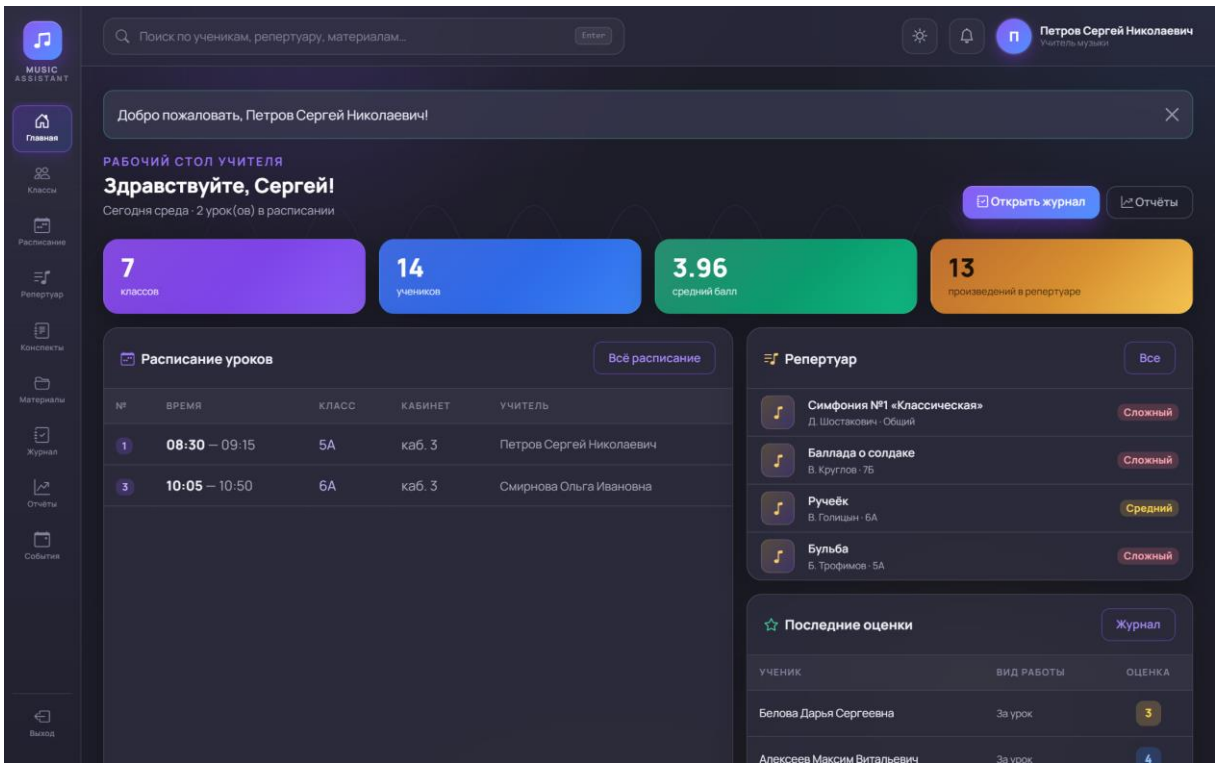


Рисунок 5 — Рабочий стол учителя

Модуль классов и учеников предоставляет справочник классов с указанием параллели, учебного года и учителя. Переход в карточку класса открывает список учеников с возможностью добавления, редактирования и удаления. Для каждого ученика можно указать родителя, дату рождения, телефон и заметки. Привязка родителя автоматически делает ученика доступным в личном кабинете соответствующего пользователя. Удаление класса каскадно удаляет его учеников, поэтому интерфейс требует подтверждения операции.

Модуль расписания формирует сетку уроков по дням недели. Урок содержит класс, учителя, предмет, номер урока, время начала и окончания, кабинет и учебный год. При сохранении проверяется корректность временного интервала: время окончания должно быть позже времени начала. Фильтр по классу позволяет быстро переключаться между параллелями. Расписание используется в личном кабинете родителя и при выборе урока в журнале.

Модуль репертуара предназначен для хранения произведений, которые используются на уроках и внеурочных занятиях. Помимо названия и композитора, указываются жанр, класс, сложность, длительность и ссылка на аудио- или видеозапись. Список поддерживает поиск по названию и композитору, а

также фильтрацию по классу и жанру. Общие произведения, не привязанные к конкретному классу, доступны всем пользователям, включая родителей.

Планы-конспекты реализованы как отдельный модуль с детальной формой. Конспект включает тему, раздел программы, цель, задачи, тип урока, класс, дату, длительность, оборудование, ход урока, домашнее задание и критерии оценивания. Такая структура соответствует требованиям к современному уроку музыки и позволяет использовать конспект как готовый методический документ. Просмотр конспекта доступен учителю, администратору и родителям классов, к которым он относится.

Модуль материалов обеспечивает загрузку файлов и добавление внешних ссылок. Поддерживаются документы, презентации, таблицы, изображения, аудиофайлы и MIDI. При загрузке проверяется расширение файла и его размер. Файл сохраняется под уникальным именем, включающим UUID, что исключает конфликты и затрудняет подбор имён злоумышленником. Исходное имя сохраняется в базе и используется при скачивании. При удалении материала связанный файл также удаляется с диска. Фрагмент проверки файла приведён в листинге 1.

Листинг 1 — Проверка и сохранение загружаемого файла (materials.py)

```
ALLOWED_EXTENSIONS = {
    'pdf', 'doc', 'docx', 'xls', 'xlsx', 'ppt', 'pptx',
    'txt', 'jpg', 'jpeg', 'png', 'gif', 'mp3', 'wav', 'ogg', 'midi', 'mid',
}

def allowed_file(filename):
    return '.' in filename and \
        filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

if file and file.filename and not allowed_file(file.filename):
    flash('Недопустимый тип файла', 'danger')
else:
    file_name = secure_filename(file.filename)
    unique_name = f"{uuid.uuid4().hex}_{file_name}"
    file.save(os.path.join(upload_dir, unique_name))
```

Журнал является центральным модулем повседневной работы учителя. Страница журнала позволяет выбрать класс, дату и урок, отметить посещаемость каждого ученика и выставить оценки по видам работ. Сохранение выполняется методом upsert: для каждой пары «ученик — урок — дата» выполняется поиск существующей записи, и при её наличии значения обновляются, а при отсутствии создаётся новая строка. Это исключает дублирование

записей при повторном сохранении формы. Фрагмент реализации приведён в листинге 2, внешний вид страницы — на рисунке 6.

Листинг 2 — Сохранение журнала без дублирования записей (journal.py)

```
for student in school_class.students:
    status = request.form.get(f'att_{student.id}')
    if status:
        row = Attendance.query.filter_by(
            student_id=student.id,
            schedule_id=schedule_id,
            lesson_date=lesson_date).first()
        if row is None:
            row = Attendance(student_id=student.id,
                             schedule_id=schedule_id,
                             lesson_date=lesson_date,
                             teacher_id=current_user.id)
            db.session.add(row)
        row.status = status
        row.note = request.form.get(f'note_{student.id}', '').strip() or None

    value = request.form.get(f'grade_{student.id}', '').strip()
    if value.isdigit() and 1 <= int(value) <= 5:
        grade = Grade.query.filter_by(
            student_id=student.id,
            schedule_id=schedule_id,
            lesson_date=lesson_date,
            category=category).first()
        if grade is None:
            grade = Grade(student_id=student.id,
                          schedule_id=schedule_id,
                          lesson_date=lesson_date,
                          category=category,
                          teacher_id=current_user.id)
            db.session.add(grade)
        grade.value = int(value)

db.session.commit()
```

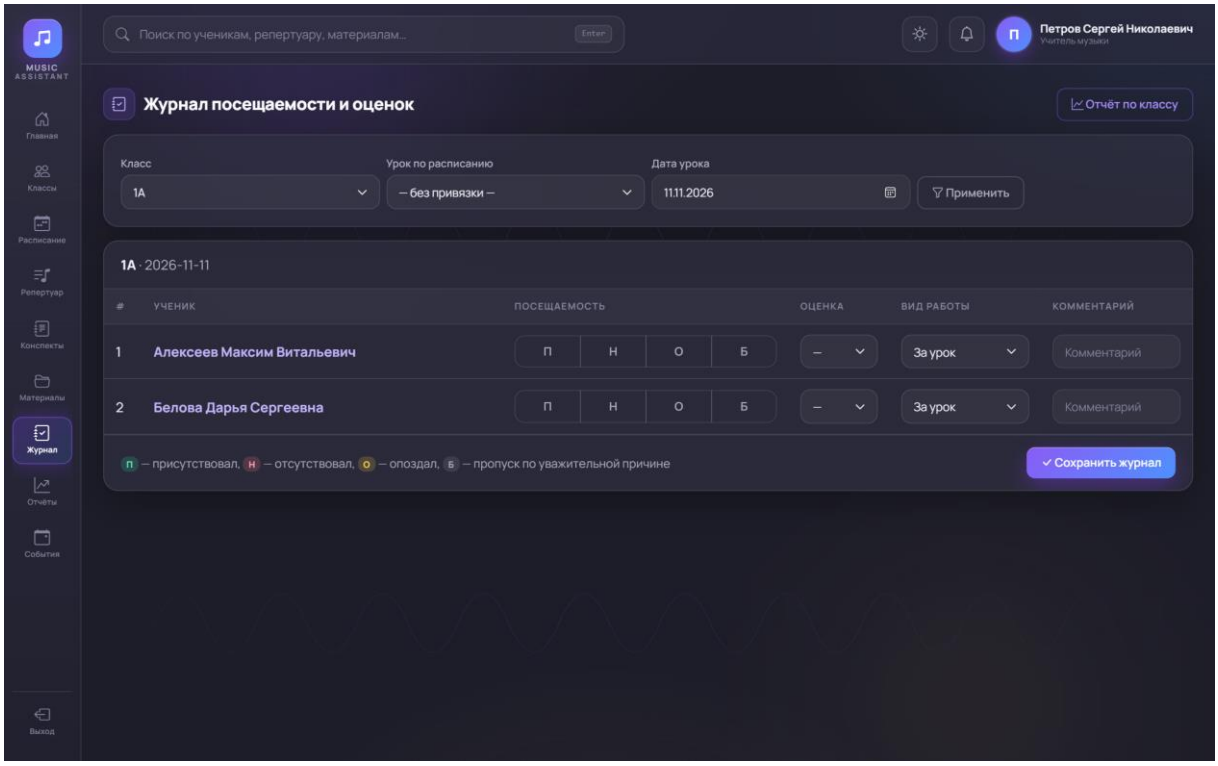


Рисунок 6 — Журнал посещаемости и оценок

Индивидуальная карточка ученика доступна из журнала и из списка класса. Она содержит сведения об ученике, средний балл, процент посещаемости, историю оценок и посещаемости. Родитель может открыть карточку только своего ребёнка; попытка открыть чужую карточку по прямому адресу возвращает ошибку 403. Карточка помогает классному руководителю и родителям увидеть динамику результатов без просмотра всего журнала.

Модуль отчётов формирует сводку по классу за выбранный период: весь учебный год, месяц или четверть. Рассчитываются количество уроков, процент посещаемости, средний балл, распределение оценок и успеваемость по каждому ученику. Предусмотрена выгрузка отчёта в CSV с разделителем «точка с запятой», что обеспечивает корректное открытие файла в русскоязычном Excel. Страница отчётов показана на рисунке 7.

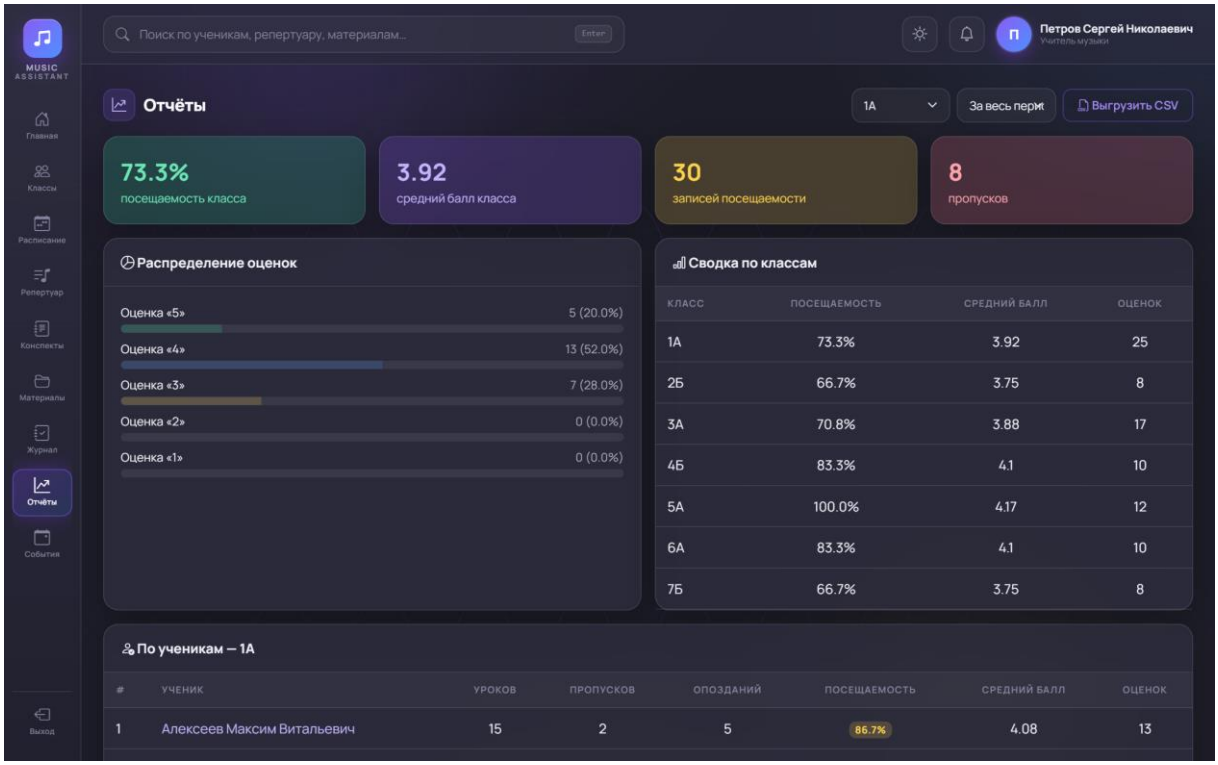


Рисунок 7 — Отчёты по классу

Модуль мероприятий ведёт афишу школьных концертов, конкурсов и музыкальных гостиных. Мероприятие содержит название, описание, дату, время, место, класс и статус (запланировано, проведено, отменено). Родители видят мероприятия своих классов и общие события. Список сортируется по дате,

что позволяет быстро определить ближайшие события и использовать афишу при планировании.

Глобальный поиск объединяет результаты по ученикам, классам, репертуару, материалам, конспектам и мероприятиям. Поисковый запрос передаётся в маршрут, который выполняет выборку по соответствующим полям и фильтрует результаты с учётом роли. Родитель видит только данные классов, в которых учатся его дети, и общие материалы. Единая строка поиска расположена в верхней панели и доступна с любой страницы.

Интерфейс приложения выполнен в стиле «тёмный концертный зал». Используются глубокий тёмный фон, деликатные фиолетовые и золотые свечения, полупрозрачные карточки с размытием, узкая вертикальная панель навигации и градиентные кнопки. Шрифт Manrope обеспечивает современный вид и хорошую читаемость. Предусмотрен переключатель светлой и тёмной темы: выбранное значение сохраняется в localStorage браузера и применяется до отрисовки страницы, что исключает мерцание. Светлая тема показана на рисунке 8.

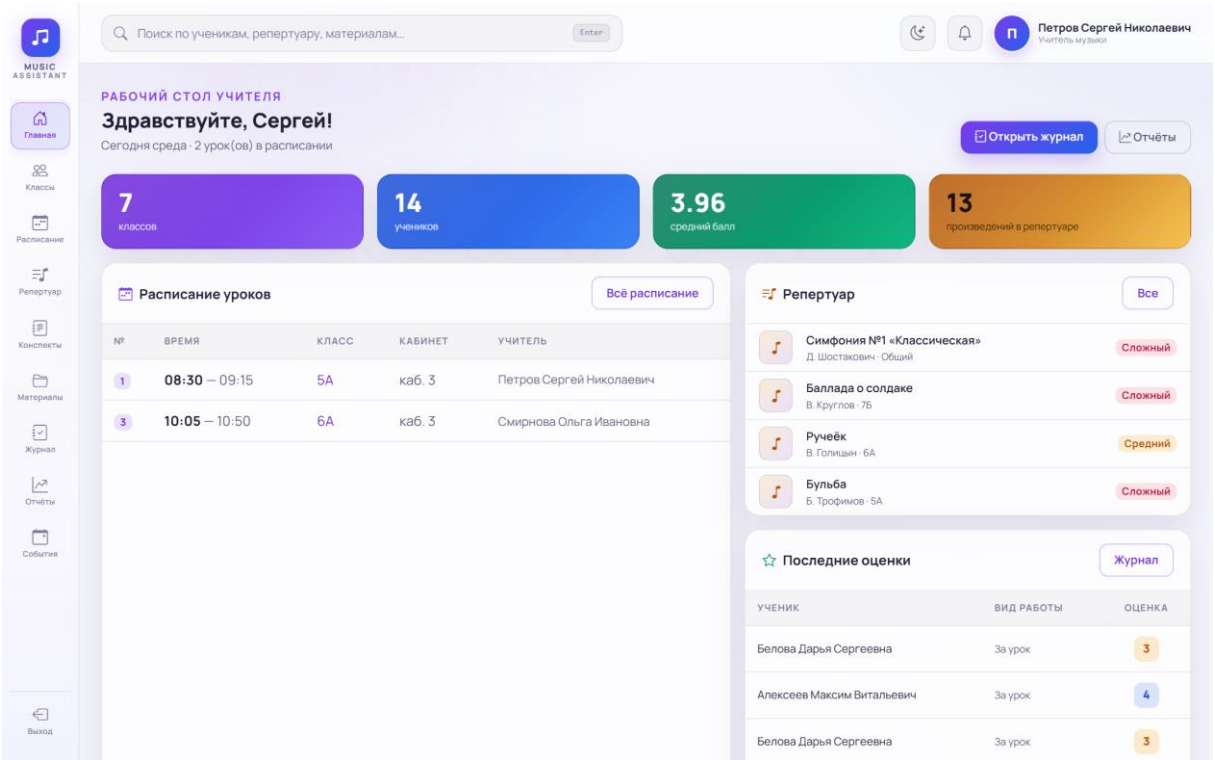


Рисунок 8 — Светлая тема интерфейса

Кабинет родителя является отдельным сценарием главной страницы. После входа родитель видит карточки своих детей с указанием класса, последних оценок и посещаемости, а также расписание и ближайшие мероприятия. Все выборки фильтруются по идентификатору родителя. Кабинет родителя представлен на рисунке 9.

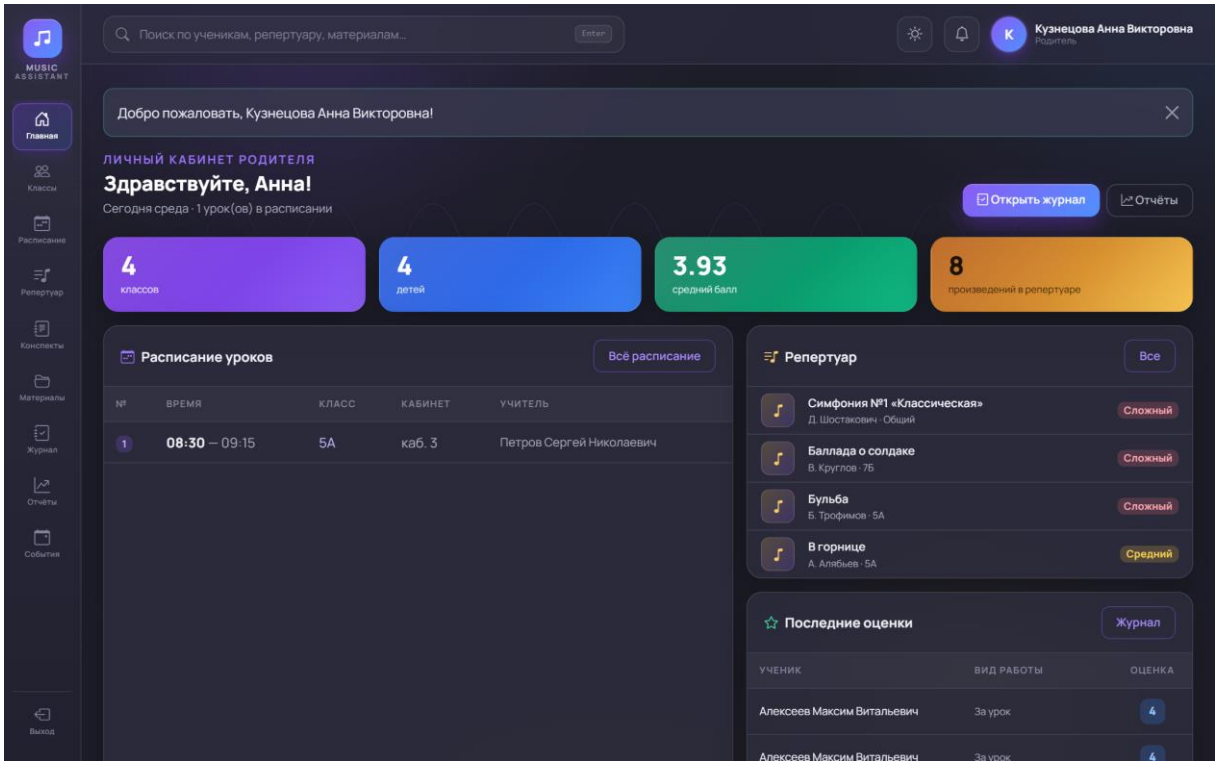


Рисунок 9 — Личный кабинет родителя

Реализация модулей следует единому шаблону: маршрут получает параметры, проверяет права, выполняет операции с моделями, формирует сообщение об успехе или ошибке и перенаправляет пользователя. Единообразие упрощает сопровождение и снижает вероятность появления несогласованных сценариев. Все формы содержат CSRF-токен, а действия изменения и удаления доступны только учителю и администратору.

2.4. Разграничение прав доступа и защита информации

Система обслуживает три категории пользователей с разными задачами и разным объёмом допустимых данных. Администратор управляет учётными записями и имеет полный доступ к данным школы. Учитель ведёт классы, расписание, репертуар, конспекты, материалы, журнал, отчёты и мероприятия. Ро-

дитель получает доступ только к сведениям о своих детях и к общим материалам. Матрица прав приведена в таблице 6.

Таблица 6 — Матрица прав доступа

Возможность	Администратор	Учитель	Родитель
Вход и выход	Да	Да	Да
Главная страница	Да	Да	личный кабинет
Классы и ученики — просмотр	Да	Да	только свои классы
Классы и ученики — изменение	Да	Да	нет
Расписание — просмотр	Да	Да	только свои классы
Расписание — изменение	Да	Да	нет
Репертуар и материалы	Да	Да	свой класс и общие
Планы-конспекты	Да	Да	свой класс и общие
Журнал — просмотр	Да	Да	только свои дети
Журнал — заполнение	Да	Да	нет
Отчёты и CSV	Да	Да	только свои дети
Мероприятия	Да	Да	свой класс и общие
Управление пользователями	Да	нет	нет
Глобальный поиск	Да	Да	только доступные данные
Смена темы	Да	Да	Да

Аутентификация построена на Flask-Login. При входе пользователь указывает логин или адрес электронной почты и пароль. Система находит учётную запись и проверяет соответствие пароля хешу, сформированному Werkzeug. При успешной проверке создаётся сессия, идентификатор пользователя сохраняется в защищённой cookie. Параметры cookie включают HttpOnly и SameSite=Lax, что снижает риск доступа к сессии из клиентских сценариев и ограничивает отправку cookie при межсайтовых запросах.

Авторизация выполняется на уровне маршрутов, а не только за счёт скрытия элементов интерфейса. Декораторы admin_required и staff_required проверяют роль текущего пользователя и возвращают ошибку 403 при недостаточных правах. Такой подход исключает обход ограничений через прямой ввод адреса. Например, страница управления пользователями доступна только адми-

нистратору, а создание и редактирование классов — администратору и учителю.

Отдельное внимание уделено защите от несанкционированного доступа к объектам по идентификатору (IDOR). Если родитель открывает журнал, отчёт или расписание с чужим class_id, маршрут проверяет, входит ли этот класс в список классов его детей. При несоответствии возвращается ошибка 403 или пустой результат. Аналогичная проверка выполняется для карточки ученика и скачивания материала. Правила доступа вынесены в модуль app/utils/access.py, что позволяет использовать их единообразно в разных маршрутах. Фрагмент проверки приведён в листинге 3.

Листинг 3 — Правила доступа к классам (access.py)

```
def parent_class_ids():
    """ID классов, в которых учатся дети текущего родителя."""
    return sorted({s.class_id for s in current_user.students})

def accessible_classes():
    """Классы, доступные текущему пользователю."""
    query = SchoolClass.query
    if current_user.is_parent:
        query = query.filter(SchoolClass.id.in (parent class ids()))
    return query.order_by(SchoolClass.grade, SchoolClass.name).all()

def class_is_accessible(class_id):
    if not current_user.is_parent:
        return True
    return class_id in parent_class_ids()
```

Защита от подделки межсайтовых запросов реализована глобально с помощью CSRFProtect. Все формы, изменяющие данные, содержат скрытое поле csrf_token. Запрос без токена или с истёкшим токеном отклоняется с кодом 400 и понятной страницей ошибки. Это защищает от ситуаций, когда сторонний сайт пытается незаметно отправить форму от имени авторизованного пользователя.

Загрузка файлов ограничена списком допустимых расширений и максимальным размером 16 МБ. Имя файла очищается функцией secure_filename, а к нему добавляется уникальный префикс UUID. Файлы сохраняются вне корня шаблонов и отдаются через маршрут скачивания с проверкой прав. При удалении материала файл удаляется с диска, что предотвращает накопление неиспользуемых данных. Ошибка превышения размера обрабатывается страницей 413.

Защита от SQL-инъекций обеспечивается использованием ORM: параметры запросов передаются через методы SQLAlchemy и не конкатенируются в строку SQL. Защита от XSS основана на автоматическом экранировании переменных в Jinja2. Секретный ключ приложения задаётся через переменную окружения и не хранится в репозитории. Пароли не выводятся в интерфейс и не записываются в журналы.

Вместе с тем следует обозначить ограничения, характерные для учебного проекта. Для промышленной эксплуатации рекомендуется включить HTTPS, настроить резервное копирование базы данных и папки загрузок, ограничить частоту попыток входа, вести аудит действий пользователей и рассмотреть двухфакторную аутентификацию для администраторов. Указанные меры выходят за рамки курсовой работы, но заложенная архитектура позволяет добавить их без существенной переработки.

Таким образом, механизм разграничения прав построен по принципу «запрещено по умолчанию»: доступ к изменяющим операциям имеют только администратор и учитель, а родитель получает исключительно чтение данных своих детей. Проверки выполняются на сервере, дублируются в интерфейсе и покрываются автоматическими тестами.

2.5. Тестирование и оценка качества разработанной системы

Для проверки работоспособности системы использовалось автоматическое тестирование на основе pytest. Каждый тест получает отдельную временную базу данных SQLite, создаваемую фикстурой, поэтому рабочая база не изменяется, а тесты можно запускать повторно в любом порядке. Тестовый набор охватывает вход и доступность страниц для трёх ролей, операции создания, редактирования и удаления, работу журнала, отчётов, загрузку файлов и защиту от CSRF.

Состав тестов приведён в таблице 7.

Таблица 7 — Состав автоматических тестов

Файл	Тестов	Что проверяется
tests/test_smoke.py	10	вход, доступность всех страниц для трёх ролей, запрет /users не-администраторам, редиректы
tests/test_crud.py	9	CRUD классов, учеников, расписания, репертуара, материалов, мероприятий и пользователей; загрузка и удаление файла; запрет опасных расширений
tests/test_modules.py	7	журнал и отсутствие дубликатов, карточка ученика, планы-конспекты, отчёты и CSV, права родителя
tests/test_http.py	1	реальный HTTP-сервер, CSRF-токен, POST без токена, обход разделов под учителем
Итого	27	

Дымовые тесты подтверждают, что все основные страницы открываются под администратором, учителем и родителем и возвращают код 200. Отдельно проверяется, что страница управления пользователями недоступна учителю и родителю (код 403), а неавторизованный пользователь перенаправляется на страницу входа. Эти тесты позволяют быстро обнаружить ошибки, возникающие при изменении навигации или регистрации Blueprints.

Тесты CRUD проверяют полный цикл работы с объектами: создание, изменение, удаление и сохранение значений в базе. Для материалов дополнительно проверяется загрузка файла, его наличие на диске, корректность скачивания и удаление файла вместе с записью. Отдельный тест пытается загрузить файл с расширением .exe и убеждается, что материал не создаётся. Тест прав родителя подтверждает, что попытка создать класс или открыть управление пользователями завершается ошибкой 403.

Тесты модулей проверяют журнал, карточку ученика, планы-конспекты, отчёты и экспорт CSV. Особое внимание уделено повторному сохранению журнала: после второго сохранения количество записей не увеличивается, а значения обновляются. Тест отчёта проверяет наличие заголовка и разделителя в CSV-файле. Тесты прав родителя подтверждают, что журнал и отчёты открываются, но сохранение запрещено, а карточка чужого ученика недоступна.

Тест HTTP-уровня поднимает реальный сервер и работает с ним через HTTP-клиент. Он получает страницу входа, извлекает CSRF-токен, проверяет, что POST без токена возвращает 400, а POST с токеном создаёт сессию. Затем

выполняется обход всех разделов под учителем и проверка отчёта. Этот тест важен потому, что проверяет поведение приложения в условиях, близких к эксплуатации, а не только через тестовый клиент Flask.

Результат запуска автоматических тестов:

Листинг 4 — Результат выполнения тестов

```
$ python -m pytest
============================= test session starts ==============================
platform win32 -- Python 3.13.x, pytest-8.x
collected 27 items

tests/test_crud.py .......                                              [ 25%]
tests/test_http.py .                                                    [ 29%]
tests/test_modules.py .......                                           [ 55%]
tests/test_smoke.py ..........                                          [100%]

============================== 27 passed in 8.4s ===============================
```

Помимо автоматических тестов проводилась ручная проверка интерфейса. Проверялись тёмная и светлая темы на десяти страницах и странице входа, сохранение выбранной темы после перезагрузки и перехода между разделами, отсутствие горизонтальной прокрутки, адаптивность при ширине экрана планшета и телефона. Отдельно проверялся кабинет родителя: отображаются четыре ребёнка, только свои классы и оценки, а попытка открыть чужой класс по прямому адресу возвращает ошибку 403. Результаты ручной проверки обобщены в таблице 8.

Таблица 8 — Результаты ручной проверки интерфейса

Проверка	Результат
Тёмная тема на десяти страницах и странице входа	выполнено
Светлая тема на десяти страницах и странице входа	выполнено
Сохранение темы после перезагрузки и перехода между страницами	выполнено
Отсутствие горизонтальной прокрутки	выполнено
Ошибки в консоли браузера	не обнаружены
Кабинет родителя: только свои дети, классы и оценки	выполнено
Попытка открыть чужой класс по прямому URL	ошибка 403
Подмена class_id в отчётах, журнале и расписании	данные не раскрываются
Мобильная вёрстка сайдбара и таблиц	выполнено

Оценка качества разработки проводилась по нескольким критериям. Функциональная полнота подтверждена покрытием требований F1–F12 и успешным прохождением всех тестов. Удобство интерфейса проверено вручную: основные действия выполняются в два-три клика, навигация единообраз-

на, сообщения об ошибках понятны. Производительность на демонстрационном наборе данных удовлетворительна: страницы формируются менее чем за секунду в локальной сети. Сопровождаемость обеспечивается модульной структурой, едиными соглашениями об именовании и наличием документации.

Опытная эксплуатация проводилась на демонстрационном наборе, включающем семь классов, учеников, расписание, репертуар, материалы, мероприятия и записи журнала. Сценарии проверялись последовательно: вход под каждой ролью, просмотр главной страницы, работа с классами, заполнение журнала, формирование отчёта, скачивание материала, поиск и выход. Все сценарии завершились ожидаемым результатом. Выявленные в процессе разработки замечания, связанные с доступом родителя к чужим классам и с сохранением журнала, были устранены и закреплены тестами.

К направлениям дальнейшего развития относятся: пагинация больших списков, уведомления о предстоящих мероприятиях, экспорт отчётов в Excel, интеграция с общешкольным электронным журналом через API, расширенная аналитика успеваемости, резервное копирование и развёртывание в контейнере. Перечисленные возможности не входят в текущую версию, но архитектура приложения готова к их поэтапному добавлению.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсовой работы была разработана информационная система «Система-помощник учителя музыки в школе», предназначенная для автоматизации учебной, методической и внеурочной деятельности учителя музыки. Система объединяет ведение классов и учеников, расписание, репертуар, планы-конспекты, материалы, журнал посещаемости и оценок, мероприятия, отчёты и глобальный поиск. Разработка выполнена в виде веб-приложения на языке Python с использованием фреймворка Flask и базы данных SQLite.

В ходе работы были решены все поставленные задачи. Изучена предметная область и нормативные требования, определяющие деятельность учителя музыки. Проведён сравнительный анализ существующих решений, который показал, что универсальные школьные системы и платформы дистанционного обучения не покрывают специализированные потребности музыкального образования. Сформулированы функциональные и нефункциональные требования, определены роли пользователей и границы системы.

Обоснован выбор технологий. Показано, что Flask обеспечивает достаточную гибкость при небольшом объёме кода, SQLAlchemy упрощает работу с базой данных, Flask-Login и Flask-WTF решают задачи аутентификации и защиты форм, а Bootstrap 5 и Jinja2 позволяют создать адаптивный и единообразный интерфейс. Использование waitress и SQLite делает развёртывание простым и не требует приобретения дополнительных лицензий.

Спроектирована архитектура приложения, основанная на разделении слоёв и использовании Blueprints. Разработана модель базы данных из десяти взаимосвязанных таблиц, обеспечивающая хранение сведений о пользователях, классах, учениках, уроках, репертуаре, материалах, мероприятиях и результатах обучения. Реализованы все функциональные модули, включая журнал с сохранением без дублирования записей, отчёты с экспортом в CSV и личный кабинет родителя.

Особое внимание уделено разграничению прав доступа и защите информации. Проверка ролей выполняется на уровне маршрутов, доступ родителей к данным ограничен собственными детьми, реализована защита от подмены идентификаторов, CSRF-атак, опасных файлов и SQL-инъекций. Пароли хранятся в виде хешей, секретный ключ задаётся через переменные окружения.

Тестирование подтвердило работоспособность системы: все 27 автоматических тестов пройдены, ручная проверка интерфейса и сценариев работы не выявила ошибок. Система апробирована на демонстрационном наборе данных и может быть использована в школе после заполнения реальными сведениями. Практическая значимость работы заключается в сокращении трудозатрат учителя на ведение документации, повышении сохранности методических материалов и обеспечении прозрачного взаимодействия с родителями.

Таким образом, цель курсовой работы достигнута, а задачи выполнены в полном объёме. Разработанное приложение представляет собой законченный программный продукт, готовый к опытной эксплуатации и дальнейшему развитию. Перспективы развития связаны с пагинацией, уведомлениями, расширенной аналитикой, интеграцией с общешкольными системами и развёртыванием в защищённой инфраструктуре образовательной организации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ И ЛИТЕРАТУРЫ

Законодательные и нормативные материалы

1. Федеральный закон от 29.12.2012 № 273-ФЗ «Об образовании в Российской Федерации» (ред. от 2025 г.).
2. Федеральный государственный образовательный стандарт основного общего образования, утверждённый приказом Минпросвещения России от 31.05.2021 № 287.
3. Профессиональный стандарт «Педагог (педагогическая деятельность в сфере дошкольного, начального общего, основного общего, среднего общего образования) (воспитатель, учитель)», утверждённый приказом Минтруда России от 18.10.2013 № 544н.
4. ГОСТ Р 7.0.100-2018. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. — Москва: Стандартинформ, 2018.
5. ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения. — Москва: Издательство стандартов, 1990.
6. ГОСТ 34.601-90. Информационная технология. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания. — Москва: Издательство стандартов, 1990.

Книги и учебные пособия

7. Лутц М. Изучаем Python: в 2 т. Т. 1. — 5-е изд. — Санкт-Петербург: Диалектика, 2020. — 832 с.
8. Лутц М. Изучаем Python: в 2 т. Т. 2. — 5-е изд. — Санкт-Петербург: Диалектика, 2020. — 720 с.
9. Гринберг М. Разработка веб-приложений на Flask. — Санкт-Петербург: Питер, 2023. — 304 с.
10. Дакетт Дж. HTML и CSS: разработка и дизайн веб-сайтов. — Москва: Эксмо, 2022. — 480 с.

11. Форсье Дж., Биссекс П., Чан У. Django: разработка веб-приложений на Python. — Санкт-Петербург: Символ-Плюс, 2019. — 456 с.
12. Кузнецов С.Д. Основы баз данных. — 2-е изд., испр. — Москва: Интернет-Университет Информационных Технологий; БИНОМ. Лаборатория знаний, 2007. — 484 с.
13. Коннолли Т., Бегг К. Базы данных. Проектирование, реализация и сопровождение. Теория и практика. — 6-е изд. — Москва: Вильямс, 2020. — 1440 с.
14. Соммервилл И. Инженерия программного обеспечения. — 10-е изд. — Москва: Вильямс, 2020. — 848 с.
15. Фаулер М. Архитектура корпоративных программных приложений. — Москва: Вильямс, 2019. — 544 с.
16. Олифер В.Г., Олифер Н.А. Компьютерные сети. Принципы, технологии, протоколы. — 6-е изд. — Санкт-Петербург: Питер, 2021. — 1008 с.

Электронные ресурсы

17. Документация Python 3. — URL: https://docs.python.org/3/ (дата обращения: 10.05.2026).
18. Flask Documentation. — URL: https://flask.palletsprojects.com/ (дата обращения: 10.05.2026).
19. SQLAlchemy Documentation. — URL: https://docs.sqlalchemy.org/ (дата обращения: 10.05.2026).
20. Flask-SQLAlchemy Documentation. — URL: https://flask-sqlalchemy.palletsprojects.com/ (дата обращения: 10.05.2026).
21. Flask-Login Documentation. — URL: https://flask-login.readthedocs.io/ (дата обращения: 10.05.2026).
22. Flask-WTF Documentation. — URL: https://flask-wtf.readthedocs.io/ (дата обращения: 10.05.2026).

23. Jinja2 Documentation. — URL: https://jinja.palletsprojects.com/ (дата обращения: 10.05.2026).
24. Bootstrap 5 Documentation. — URL: https://getbootstrap.com/docs/5.3/ (дата обращения: 10.05.2026).
25. pytest Documentation. — URL: https://docs.pytest.org/ (дата обращения: 10.05.2026).
26. waitress Documentation. — URL: https://docs.pylonsproject.org/projects/waitress/ (дата обращения: 10.05.2026).
27. SQLite Documentation. — URL: https://www.sqlite.org/docs.html (дата обращения: 10.05.2026).
28. MDN Web Docs. — URL: https://developer.mozilla.org/ru/ (дата обращения: 10.05.2026).
29. Репозиторий проекта «Система-помощник учителя музыки в школе». — URL: https://github.com/Vallix9/music-teacher-assistant (дата обращения: 10.05.2026).

ПРИЛОЖЕНИЕ А

Скриншоты интерфейса

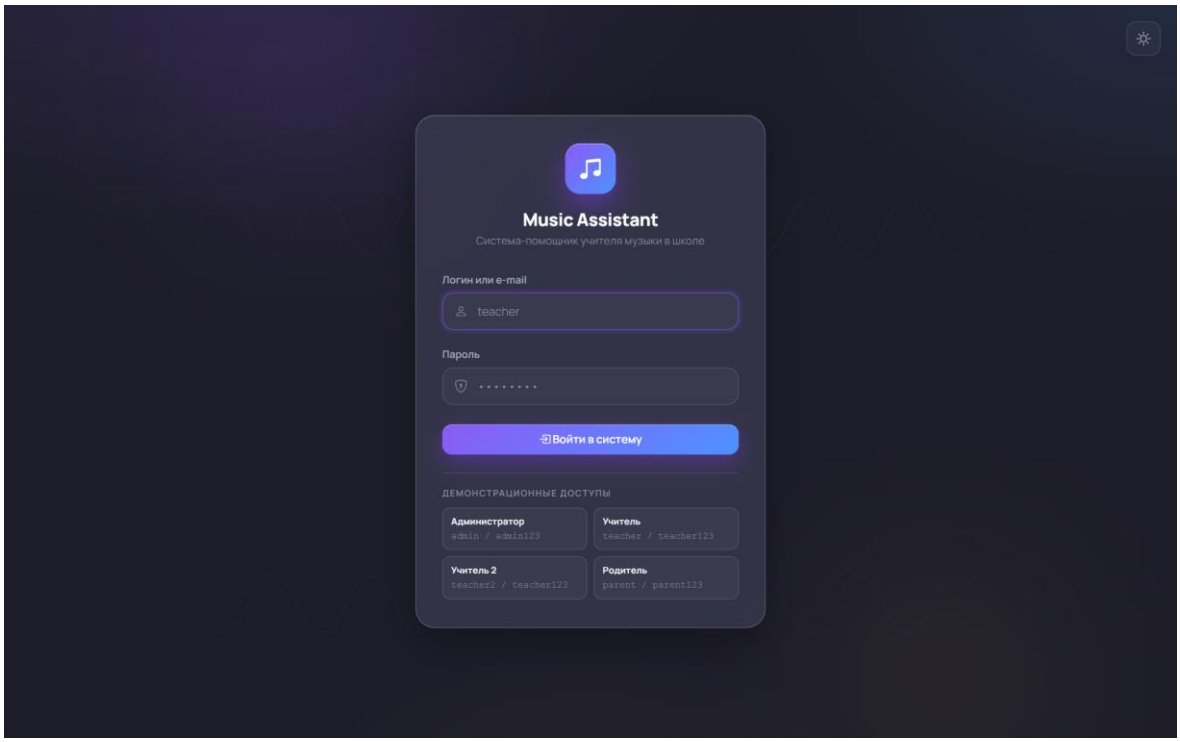


Рисунок А.1 — Страница входа в систему

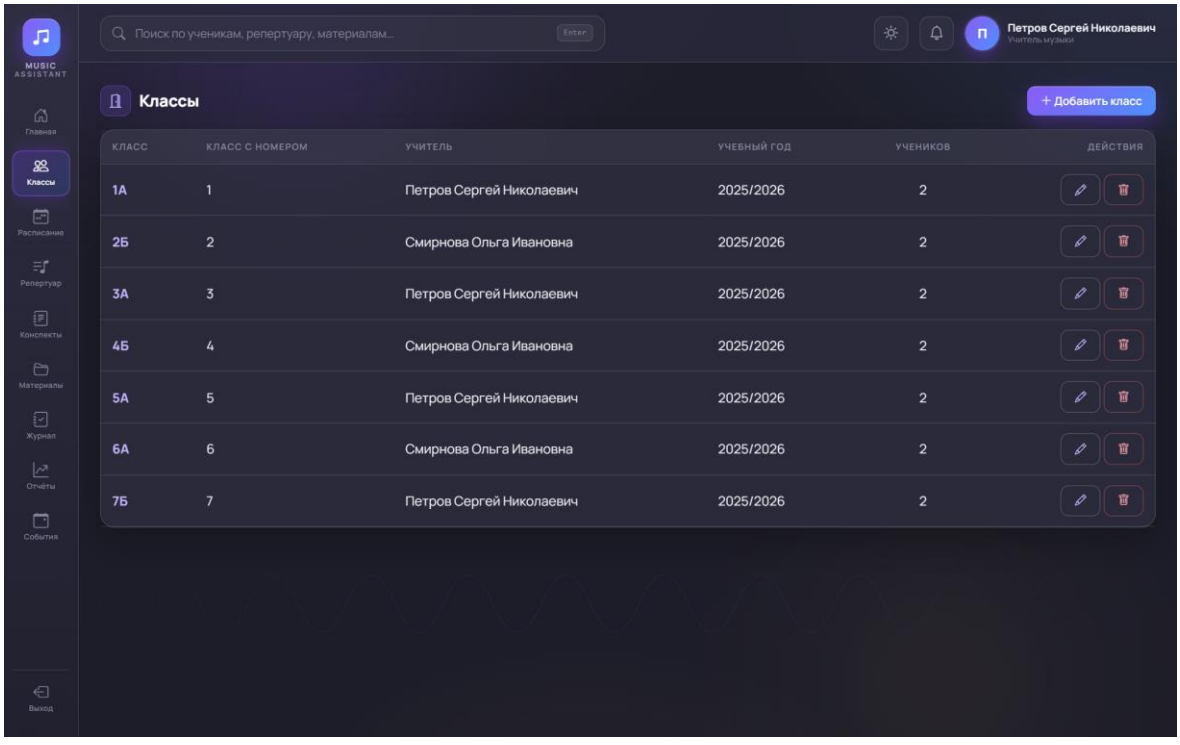


Рисунок А.2 — Список классов

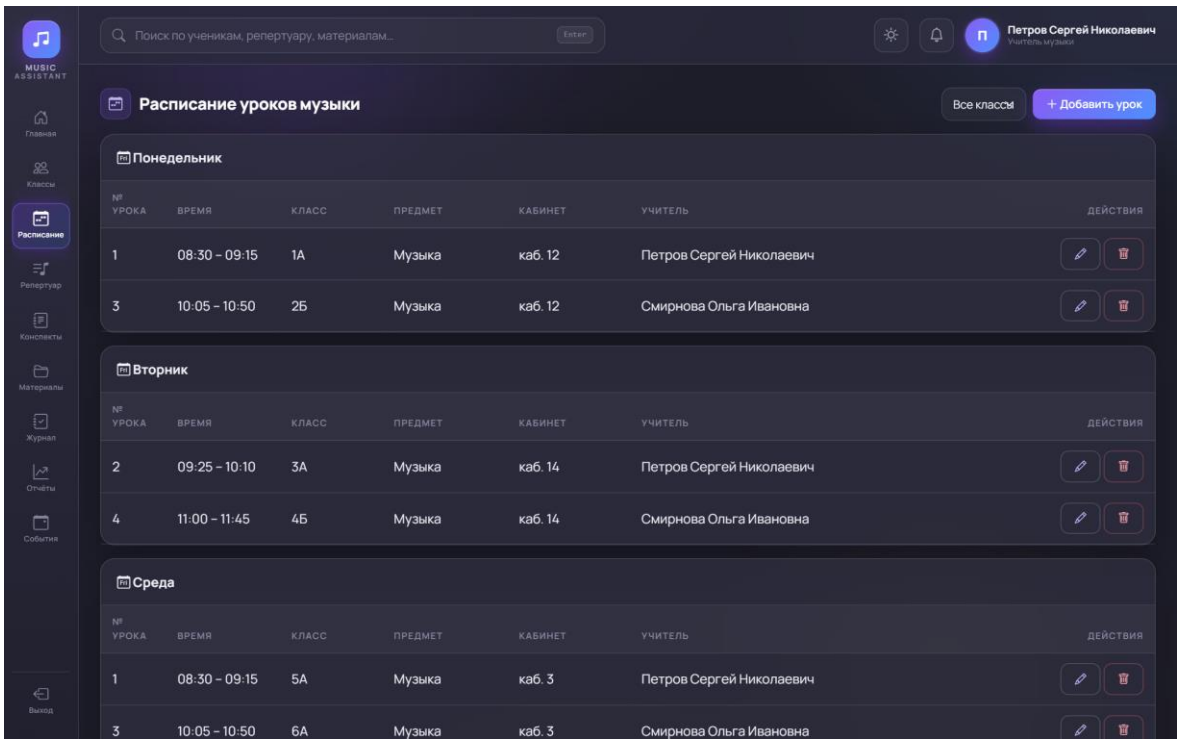


Рисунок А.3 — Расписание уроков

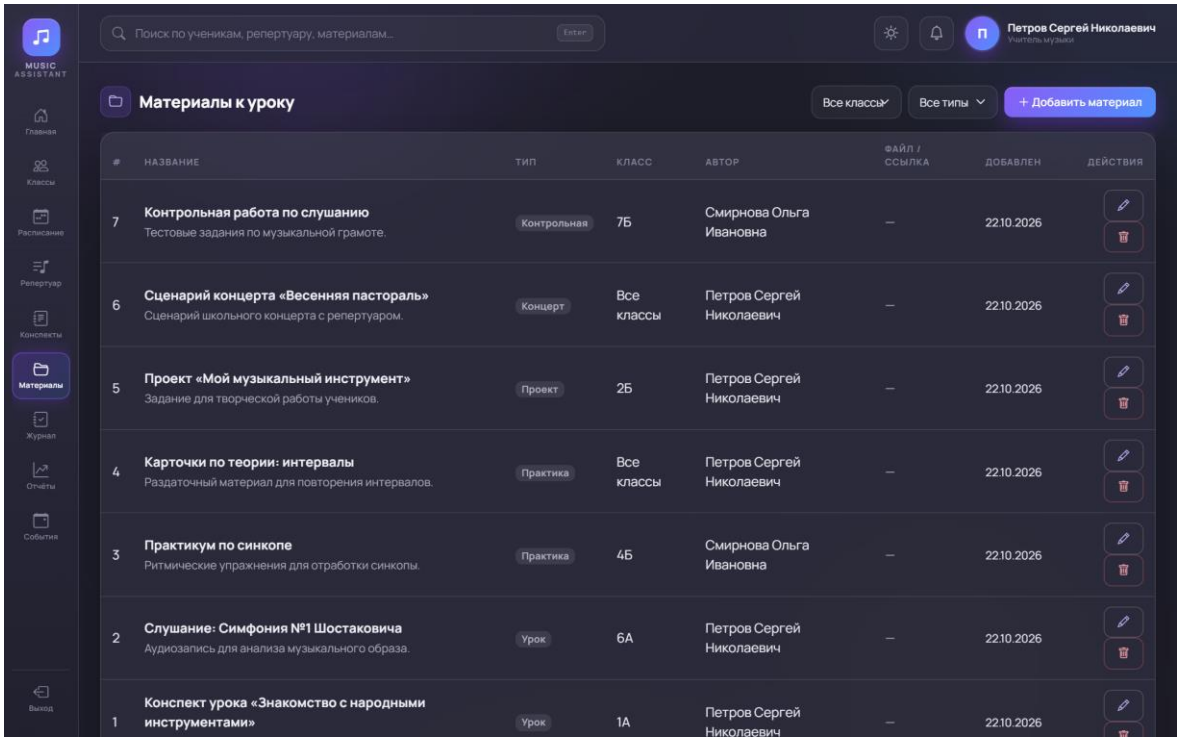


Рисунок А.4 — Материалы к урокам

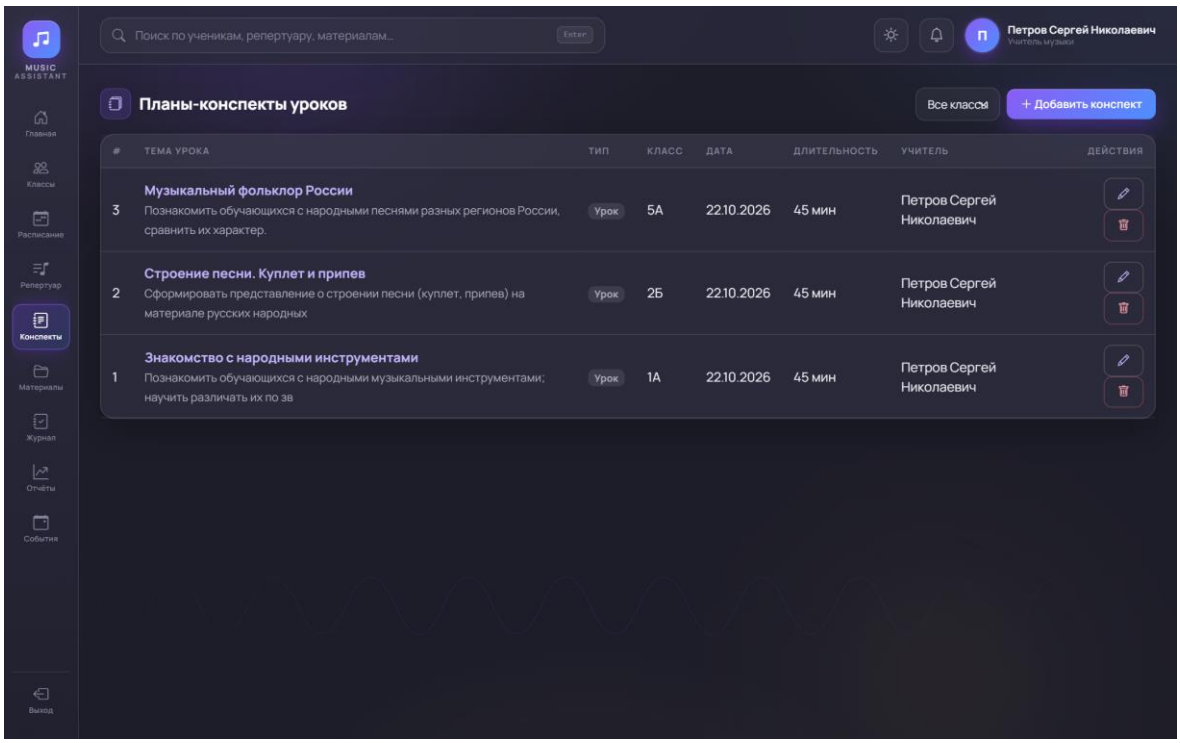


Рисунок А.5 — Планы-конспекты уроков

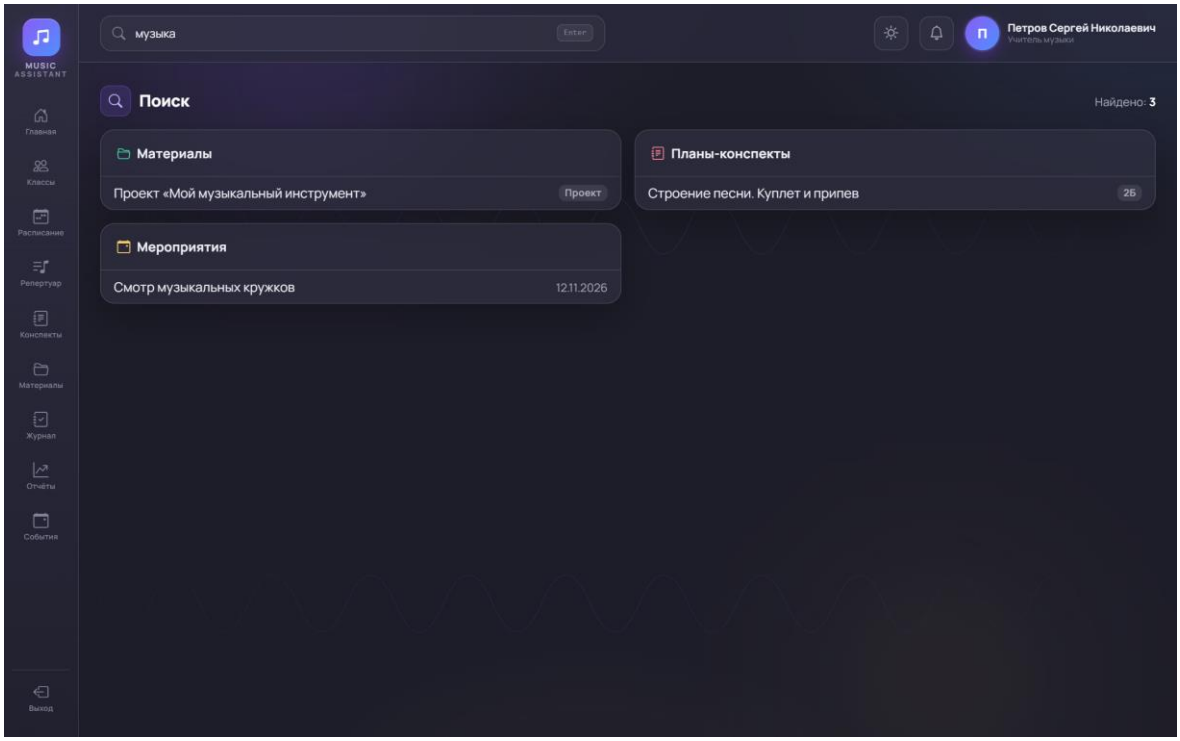


Рисунок А.6 — Результаты глобального поиска

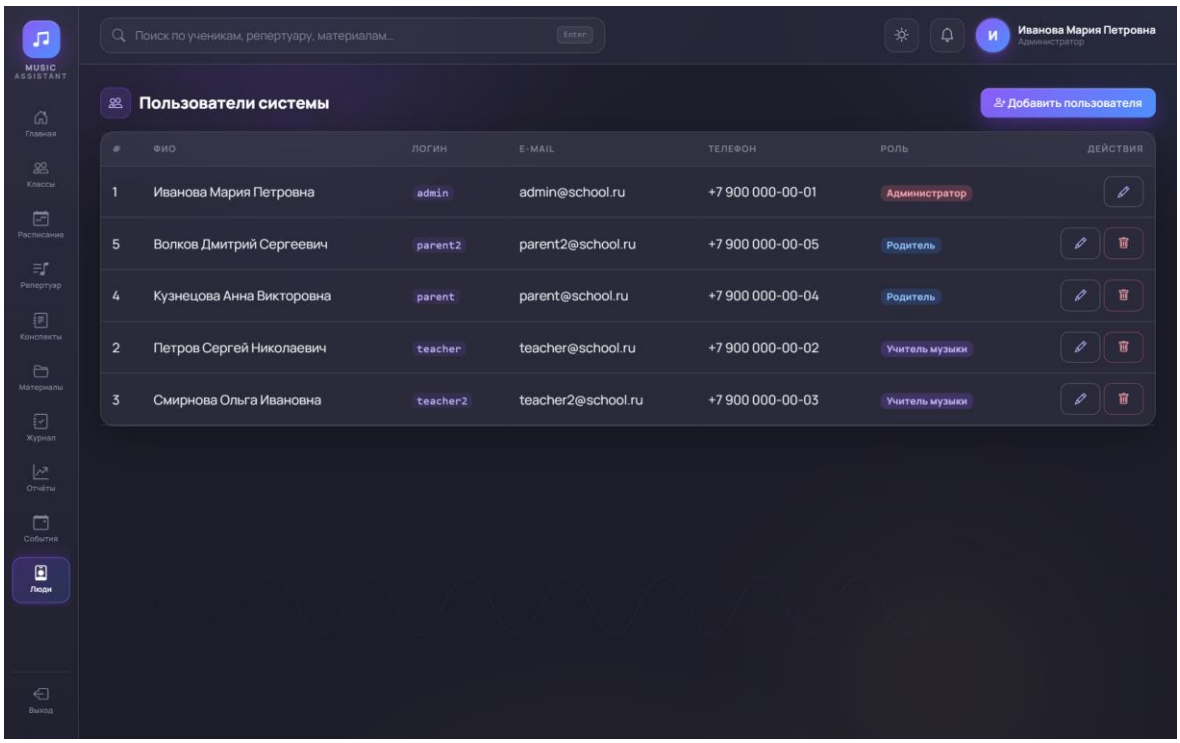


Рисунок А.7 — Управление пользователями

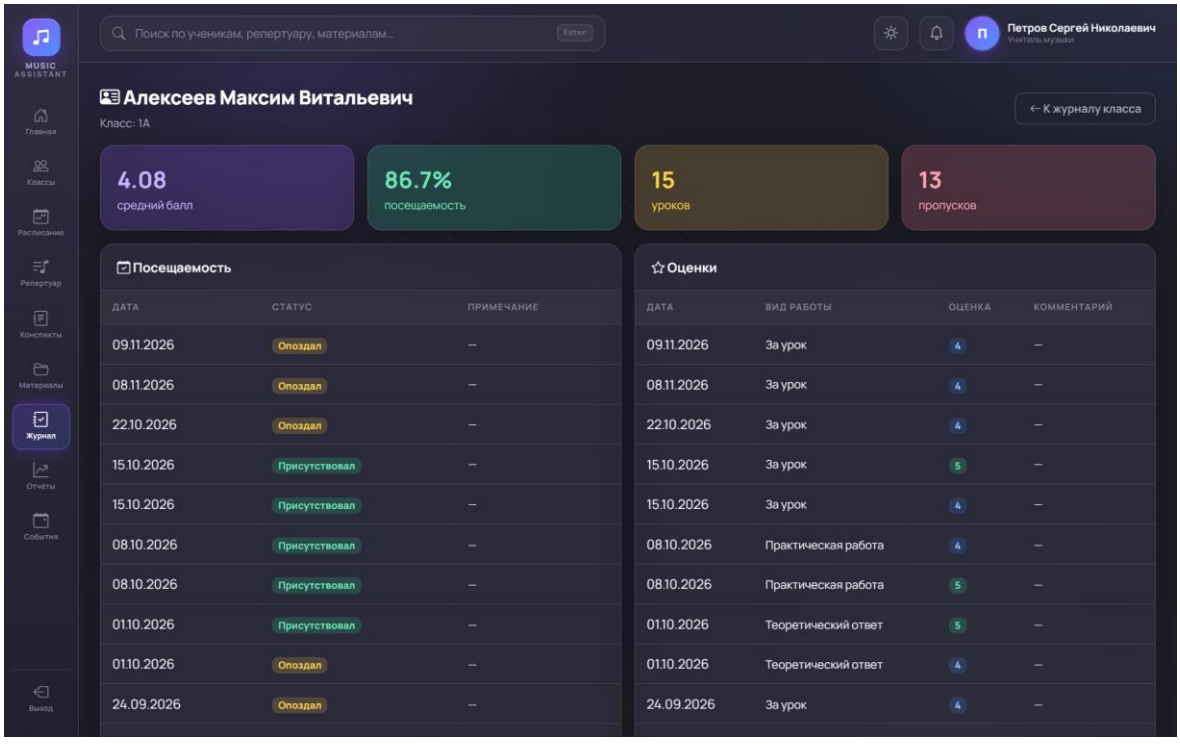


Рисунок А.8 — Карточка ученика

ПРИЛОЖЕНИЕ Б

Фрагменты исходного кода

В приложении приведены фрагменты кода, иллюстрирующие ключевые решения: фабрику приложения, регистрацию Blueprints и обработчиков ошибок, а также маршрут формирования отчёта.

Листинг Б.1 — Фабрика приложения и регистрация модулей (app/__init__.py)

```
def create_app(config_object=None):
    app = Flask(__name__)
    app.config.from_object(config_object or Config)

    db.init_app(app)
    login_manager.init_app(app)
    csrf.init_app(app)

    from app.routes.auth import auth_bp
    from app.routes.main import main_bp
    from app.routes.students import students_bp
    from app.routes.schedule import schedule_bp
    from app.routes.repertoire import repertoire_bp
    from app.routes.lesson_plans import lesson_plans_bp
    from app.routes.materials import materials_bp
    from app.routes.journal import journal_bp
    from app.routes.reports import reports_bp
    from app.routes.events import events bp
    from app.routes.search import search_bp

    for blueprint in (auth_bp, main_bp, students_bp, schedule_bp,
                      repertoire_bp, lesson_plans_bp, materials_bp,
                      journal_bp, reports_bp, events_bp, search_bp):
        app.register_blueprint(blueprint)

    @app.errorhandler(400)
    def bad_request(error):
        return render_template('errors/error.html', code=400,
                               message='Форма устарела. Обновите страницу.'), 400

    @app.errorhandler(403)
    def forbidden(error):
        return render_template('errors/error.html', code=403,
                               message='Недостаточно прав для выполнения действия.'), 403

    return app
```

Листинг Б.2 — Формирование отчёта по классу (reports.py)

```
@reports_bp.route('/reports')
@login_required
def reports():
    classes = accessible_classes()
    class_id = request.args.get('class_id', type=int)
    period = request.args.get('period', 'year')

    if not class_id and classes:
        class_id = classes[0].id
    school_class = db.session.get(SchoolClass, class_id) if class_id else None
    if school_class and school_class.id not in [c.id for c in classes]:
        school class = None

    start_date, end_date = period_bounds(period)
    rows = build_report(school_class, start_date, end_date)
    summary = summarize(rows)
    return render_template('reports/reports.html', classes=classes,
                           school class=school class, rows=rows,
                           summary=summary, period=period)
```

ПРИЛОЖЕНИЕ В

Руководство по развёртыванию

Для запуска системы на компьютере под управлением Windows необходимо установить Python версии 3.10 или выше. Затем следует открыть командную строку в папке проекта, создать виртуальное окружение и установить зависимости:

Листинг В.1 — Команды установки и запуска

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python app.py
```

После запуска приложение доступно по адресу http://127.0.0.1:5000. При первом обращении автоматически создаётся и заполняется демонстрационными данными база данных (файл instance/music_assistant.db). Для входа можно использовать учётные записи: администратор — admin / admin123, учитель — teacher / teacher123, родитель — parent / parent123. Пароли демонстрационных учётных записей следует изменить перед реальной эксплуатацией.

Параметры запуска задаются переменными окружения HOST, PORT и FLASK_DEBUG. Для перехода на MySQL необходимо установить драйвер PyMySQL (он уже включён в requirements.txt) и указать строку подключения в переменной DATABASE_URL. Резервное копирование сводится к копированию файла базы данных и папки app/static/uploads.